

Beej's Guide to C — Companion Exercise Sheet: Solutions

Reference answers, chapter by chapter

Before you read this

Chapter 2 — Hello, World!

Chapter 3 — Variables and Statements

Chapter 4 — Functions

Chapter 5 — Pointers

Chapter 6 — Arrays

Chapter 7 — Strings

Chapter 8 — Structs

Chapter 9 — File Input/Output

Chapter 10 — typedef

Chapter 11 — Pointers II: Arithmetic

Chapter 12 — Manual Memory Allocation

Chapter 13 — Scope

Chapter 14 — Types II

Chapter 15 — Types III: Conversions

Chapter 16 — Types IV: Qualifiers and Specifiers

Chapter 17 — Multifile Projects

Chapter 18 — The Outside Environment

Chapter 19 — The C Preprocessor

Chapter 20 — structs II

Chapter 21 — Characters and Strings II

Chapter 22 — enum

Chapter 23 — Pointers III

Chapter 24 — Bitwise Operations

Chapter 25 — Variadic Functions

Chapter 26 — Locale and Internationalization

Chapter 27 — Unicode, Wide Characters

Chapter 28 — Exiting a Program

Chapter 29 — Signal Handling

Chapter 30 — Variable-Length Arrays

Chapter 31 — goto

Chapter 32 — Compound Literals and Generic Selections

Chapter 33 — Arrays Part II

Chapter 34 — setjmp, longjmp

Chapter 35 — Incomplete Types

Chapter 36 — Complex Numbers

Chapter 37 — Fixed Width Integer Types

Chapter 38 — Date and Time

Chapter 39 — Multithreading

Chapter 40 — Atomics

Chapter 41 — Function Specifiers and Alignment

Milestone Project Notes

Before you read this

Every solution here has been compiled and run clean under GCC with warnings and sanitizers enabled:

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined -o program program.c
```

You don't need all of that to follow along — the exercise sheet introduces those flags one at a time, and `gcc -o program program.c` is enough for the early chapters. A few solutions need `-lm` (anything using `math.h` or `complex.h`) or `-pthread` (Chapters 39–40); those say so where they appear.

Three solutions deliberately produce a compiler warning, and one deliberately aborts at runtime. Each is labelled — the warning is the lesson in those cases, so don't “fix” it.

They are *reference* answers, not the only right answers. If your version differs but is correct and warning-free, yours is fine. Where a solution makes a non-obvious choice, there's a note explaining why — read those; they're the part that actually teaches.

Exercises that are “run this and write down what happened” have discussion instead of code.

Every solution uses only what Beej has covered by that chapter. The few unavoidable exceptions are marked *[ahead: Ch N]* — a cast, an `unsigned char`, a `const` qualifier. Copy those and keep going; the explanation lands in that chapter.

Chapter 2 — Hello, World!

2.1

```
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```

`int main(void)` rather than `int main()`: the `void` explicitly says “no parameters”. Empty parens historically meant “unspecified parameters”, which disables argument checking. In C23 the two finally mean the same thing, but writing `void` is still the habit to have.

2.2

Expected first errors (wording varies by compiler):

Break	Typical first error
No <code>#include <stdio.h></code>	implicit declaration of function ‘printf’
Missing semicolon	expected ‘;’ before ‘return’
<code>Main</code> instead of <code>main</code>	linker error: undefined reference to ‘main’
Missing closing brace	expected declaration or statement at end of input

The lesson: the missing-brace error points at the *end of the file*, nowhere near the actual mistake, and the `Main` error comes from the **linker**, not the compiler — it's a different tool with different error formatting. Learning to tell those apart saves hours later.

2.3

```

#include <stdio.h>

int main(void)
{
    printf("__STDC_VERSION__ = %ld\n", __STDC_VERSION__);
    return 0;
}

```

Roughly: C99 gives 199901, C11 gives 201112, C17 gives 201710, C23 gives 202311. Under `-std=c89` the macro isn't defined at all and this fails to compile — which is itself the answer to “how do I detect C89?”.

Chapter 3 — Variables and Statements

3.1

```

#include <stdio.h>

int main(void)
{
    printf("%8s %10s\n", "Celsius", "Fahrenheit");
    for (int c = -40; c <= 100; c += 10) {
        double f = c * 9.0 / 5.0 + 32.0;
        printf("%8d %10.1f\n", c, f);
    }
    return 0;
}

```

`9.0 / 5.0` not `9 / 5`. The integer version evaluates to 1 and quietly ruins every row.

3.2

```

#include <stdio.h>

int main(void)
{
    /* Version 1: if/else chain – the one to ship */
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0)    printf("FizzBuzz\n");
        else if (i % 3 == 0) printf("Fizz\n");
        else if (i % 5 == 0) printf("Buzz\n");
        else                printf("%d\n", i);
    }

    /* Version 2: switch on i % 15 */
    for (int i = 1; i <= 100; i++) {
        switch (i % 15) {
            case 0:
                printf("FizzBuzz\n");
                break;
            case 3: case 6: case 9: case 12:
                printf("Fizz\n");
                break;
            case 5: case 10:
                printf("Buzz\n");
                break;
            default:
                printf("%d\n", i);
                break;
        }
    }

    /* Version 3: chained ternary */
    for (int i = 1; i <= 100; i++) {
        i % 15 == 0 ? printf("FizzBuzz\n") :
        i % 3 == 0 ? printf("Fizz\n") :
        i % 5 == 0 ? printf("Buzz\n") :
                    printf("%d\n", i);
    }

    return 0;
}

```

Version 1 is the one to ship. It reads like the rules do.

Version 2 works because `i % 15` collapses the problem to 15 possible cases. The grouped labels (`case 3: case 6:`) rely on fall-through: an empty case body falls into the next one, so all four land on the same `printf`. That's the one legitimate use of fall-through — a `break` is still required at the end of each group, and leaving one out by accident is a classic bug (`-Wimplicit-fallthrough` catches it).

Version 3 compiles because `printf` returns an `int`, so each branch of the ternary has a usable type. It's a demonstration that the ternary is an *expression*, not a statement — and also a demonstration that being clever about it makes the code worse. Nobody reading this in six months will thank you.

Note what none of these do: build the output string first. That would need `snprintf`, which is Chapter 9. Everything here is `printf`, `%`, `for`, `if`, `switch`, and `?:` — all Chapter 3 or earlier.

3.3

```
int i = 5;
printf("%d %d\n", i++, i); /* UNSPECIFIED: no sequence point between args */

int j = 5;
printf("%d\n", j++ + ++j); /* UNDEFINED: j modified twice, no sequence point */
```

The first line is *unspecified* — the compiler may evaluate the arguments in any order, so you might get 5 6 or 5 5. The second is genuinely *undefined behaviour*: `j` is modified twice between sequence points, and the standard imposes no requirement at all on what the program does. Not “picks one answer” — no requirement. The compiler may assume it never happens and optimize accordingly.

The rule to internalize: never modify the same object twice in one expression, and never read an object in an expression that also modifies it, except through the operators that impose ordering (`&&`, `||`, `?:`, `,`).

3.4

```
#include <stdio.h>

int main(void)
{
    int n = 91827;

    int sign = n < 0 ? -1 : 1;
    int v = n * sign; /* work with the positive value */

    int sum = 0;
    int rev = 0;
    while (v > 0) {
        int d = v % 10;
        sum += d;
        rev = rev * 10 + d;
        v /= 10;
    }

    printf("n = %d\n", n);
    printf("digit sum: %d\n", sum);
    printf("reversed: %d\n", rev * sign);
    return 0;
}
```

The `sign` variable is how you handle negatives without a separate code path: strip the sign, do the arithmetic on a positive number, put the sign back at the end. Trying to run the loop on a negative `n` directly gives you negative digits, because in C the result of `%` takes the sign of the left operand — `-91827 % 10` is `-7`, not `3`.

One limitation worth knowing about, though you can’t fix it with Chapter 3 material: `n = INT_MIN` breaks this. That value has no positive counterpart in an `int`, so `n * sign` overflows. Chapter 14 explains why, and Chapter 15 gives you the tools (a wider type) to deal with it.

Chapter 4 — Functions

4.1

```

#include <stdio.h>

int max3(int a, int b, int c);
int min3(int a, int b, int c);

int main(void)
{
    printf("%d %d\n", max3(3, 9, 4), min3(3, 9, 4));
    return 0;
}

int max3(int a, int b, int c)
{
    int m = a > b ? a : b;
    return m > c ? m : c;
}

int min3(int a, int b, int c)
{
    int m = a < b ? a : b;
    return m < c ? m : c;
}

```

Delete the prototypes and you get “implicit declaration of function” — in C99 and later that’s a constraint violation, and with `-Wall` most compilers make it an error outright.

4.2

```

void swap(int a, int b) /* does nothing useful */
{
    int t = a;
    a = b;
    b = t;
}

```

C passes everything by value. `a` and `b` are copies living in this function’s frame; assigning to them modifies the copies and nothing else. There is no pass-by-reference in C — the pointer version in 5.2 simulates it by passing the *address* by value.

4.3

```

#include <stdio.h>

int factorial_rec(int n) { return n <= 1 ? 1 : n * factorial_rec(n - 1); }

int factorial_itr(int n)
{
    int r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}

int fib_rec(int n) { return n < 2 ? n : fib_rec(n - 1) + fib_rec(n - 2); }

int fib_itr(int n)
{
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        int t = a + b;
        a = b;
        b = t;
    }
    return a;
}

int main(void)
{
    printf("%d %d\n", factorial_rec(12), factorial_itr(12));
    printf("%d %d\n", fib_rec(35), fib_itr(35));
    return 0;
}

```

12! is 479,001,600, which just fits in a 32-bit `int`. 13! does not, and you get a silently wrong answer — an early taste of Chapter 14.

Recursive `fib` is exponential: it recomputes the same subproblems over and over, roughly `fib(n)` times. Run the two with time `./program` and `fib(35)` takes something on the order of a second while the iterative version is instantaneous. `fib(45)` recursive will keep your machine busy for minutes. Cheapest possible demonstration that algorithmic complexity beats micro-optimization.

4.4

```

#include <stdio.h>
#include <stdbool.h>

bool is_prime(int n)
{
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (int i = 3; i * i <= n; i += 2)
        if (n % i == 0) return false;
    return true;
}

int gcd(int a, int b)
{
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a < 0 ? -a : a;
}

bool is_perfect(int n)
{
    if (n < 2) return false;
    int sum = 1;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            sum += i;
            if (i != n / i) sum += n / i;
        }
    }
    return sum == n;
}

int main(void)
{
    printf("perfect under 10000:");
    for (int i = 2; i < 10000; i++)
        if (is_perfect(i)) printf(" %d", i);
    printf("\n");

    printf("twin primes under 200:");
    for (int i = 2; i < 200; i++)
        if (is_prime(i) && is_prime(i + 2)) printf(" (%d,%d)", i, i + 2);
    printf("\n");
    printf("gcd(48,18) = %d\n", gcd(48, 18));
    return 0;
}

```

`i * i <= n` instead of `i <= sqrt(n)`: no floating point, no `-lm`, no precision worry at the boundary. The perfect-number loop pairs each divisor with `n / i`, so it's $O(\sqrt{n})$ instead of $O(n)$ — and the `i != n / i` guard stops a perfect square's root being counted twice.

Chapter 5 — Pointers

5.1

```

#include <stdio.h>

int main(void)
{
    int i = 1;
    int i2 = 2;
    char c = 'x';
    double d = 3.14;

    printf("int    at %p size %zu\n", (void *)&i, sizeof i);
    printf("int    at %p size %zu\n", (void *)&i2, sizeof i2);
    printf("char   at %p size %zu\n", (void *)&c, sizeof c);
    printf("double at %p size %zu\n", (void *)&d, sizeof d);

    return 0;
}

```

[ahead: Ch15] The `(void *)` cast is there because `%p` is specified to take a `void *`; passing an `int *` happens to work everywhere but isn't strictly correct. Casts are Chapter 15 — copy it for now.

The two `ints` may or may not come out adjacent, and the answer can change when you recompile with different optimization settings. The compiler orders and aligns local variables however it likes, and nothing in the standard says otherwise. Arrays are the exception — their elements *are* guaranteed contiguous, which is what makes Chapter 6 and Chapter 11 work.

5.2

```

#include <stdio.h>

void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

void swap3(int *a, int *b, int *c)
{
    int t = *a;
    *a = *b;
    *b = *c;
    *c = t;
}

int main(void)
{
    int x = 1, y = 2, z = 3;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    swap3(&x, &y, &z);
    printf("%d %d %d\n", x, y, z);

    return 0;
}

```

5.3

```

#include <stdio.h>
#include <stdbool.h>

bool divmod(int num, int den, int *quot, int *rem)
{
    if (den == 0) return false;
    *quot = num / den;
    *rem = num % den;
    return true;
}

int main(void)
{
    int q, r;
    if (divmod(17, 5, &q, &r)) printf("%d rem %d\n", q, r);
    if (!divmod(17, 0, &q, &r)) printf("division by zero rejected\n");
    return 0;
}

```

The `bool` version is better. The `void` version leaves the caller with no way to distinguish “it failed” from “it succeeded and the answer happened to be whatever was already in those variables”. This is the standard C idiom: **return a status, deliver results through out-parameters**.

5.4

```

#include <stdio.h>

void safe_increment(int *p)
{
    if (p != NULL) (*p)++;
}

int main(void)
{
    int x = 5;
    safe_increment(&x);
    safe_increment(NULL);
    printf("%d\n", x);
    return 0;
}

```

Note `(*p)++` and not `*p++`. The latter parses as `*(p++)` — it increments the *pointer* and discards the dereferenced value. Precedence bug, extremely common.

Without the NULL check, ASan reports `SEGV on unknown address 0x000000000000` and gives you a stack trace pointing at the exact line. Get used to reading those reports now; they’re far more informative than a bare “Segmentation fault”.

Chapter 6 — Arrays

6.1

```

#include <stdio.h>

int main(void)
{
    int arr[] = {42, 7, -3, 19, 88, 0, 15};
    size_t len = sizeof arr / sizeof arr[0];

    int min = arr[0], max = arr[0], sum = 0;
    for (size_t i = 0; i < len; i++) {
        if (arr[i] < min) min = arr[i];
        if (arr[i] > max) max = arr[i];
        sum += arr[i];
    }
    printf("len %zu min %d max %d sum %d mean %.2f\n",
        len, min, max, sum, (double)sum / len);
    return 0;
}

```

`(double)sum / len` forces floating-point division; without the cast you'd get integer division and a truncated mean. *[ahead: Ch15]* — casts are Chapter 15, but this one is unavoidable here.

`sizeof arr / sizeof arr[0]` only works where `arr` is a real array. Pass it to a function and it decays to a pointer, so the same expression silently gives you `8 / 4 == 2`. That trap is Exercise 19.3.

6.2

```

#include <stdio.h>

void reverse(int *arr, int len)
{
    for (int i = 0, j = len - 1; i < j; i++, j--) {
        int t = arr[i];
        arr[i] = arr[j];
        arr[j] = t;
    }
}

void rotate(int *arr, int len, int k)
{
    if (len <= 0) return;
    k %= len;
    if (k < 0) k += len; /* normalize negative rotation */
    reverse(arr, k);
    reverse(arr + k, len - k);
    reverse(arr, len);
}

int main(void)
{
    int a[] = {1, 2, 3, 4, 5, 6, 7};
    rotate(a, 7, 10); /* same as rotating by 3 */
    for (int i = 0; i < 7; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}

```

The three-reversal trick: reverse the first `k`, reverse the rest, reverse the whole thing. $O(n)$ time, $O(1)$ space, and no temporary buffer.

6.3

```
#include <stdio.h>

void print_matrix(int m[4][4])
{
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) printf("%4d", m[i][j]);
        printf("\n");
    }
}

void transpose(int m[4][4])
{
    for (int i = 0; i < 4; i++)
        for (int j = i + 1; j < 4; j++) {
            int t = m[i][j];
            m[i][j] = m[j][i];
            m[j][i] = t;
        }
}

int main(void)
{
    int m[4][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,16}};
    print_matrix(m);
    transpose(m);
    printf("\ntransposed:\n");
    print_matrix(m);

    for (int i = 0; i < 4; i++) {
        int rs = 0, cs = 0;
        for (int j = 0; j < 4; j++) { rs += m[i][j]; cs += m[j][i]; }
        printf("row %d sum %d, col %d sum %d\n", i, rs, i, cs);
    }
    return 0;
}
```

`int m[N][N]` as a parameter really means `int (*m)[N]` — a pointer to an array of `N` ints. That's why the *second* dimension is mandatory and the first isn't: the compiler needs the row stride to compute `m[i][j]`.

The literal `4` repeated everywhere is exactly the kind of thing `#define` exists to fix — that's Chapter 19.

The inner loop starting at `j = i + 1` is the whole trick to transposing in place — starting at 0 swaps every pair twice and gives you back the original matrix.

6.4

```

#include <stdio.h>
#include <stdbool.h>

int main(void)
{
    bool composite[1000] = {false};
    for (int i = 2; i * i < 1000; i++)
        if (!composite[i])
            for (int j = i * i; j < 1000; j += i)
                composite[j] = true;

    int count = 0, prev = 0, max_gap = 0, gap_at = 0;
    for (int i = 2; i < 1000; i++) {
        if (composite[i]) continue;
        printf("%5d", i);
        if (++count % 10 == 0) printf("\n");
        if (prev && i - prev > max_gap) { max_gap = i - prev; gap_at = prev; }
        prev = i;
    }
    printf("\n%d primes, largest gap %d after %d\n", count, max_gap, gap_at);
    return 0;
}

```

Inner loop starts at `i * i`, not `2 * i`: every smaller multiple of `i` already got crossed off by a smaller prime factor. Outer loop stops at `i * i < LIMIT` for the same reason.

Chapter 7 — Strings

7.1

```

#include <stdio.h>
#include <string.h>

size_t my_strlen(char *s)
{
    size_t i = 0;
    while (s[i] != '\0') i++;
    return i;
}

void my_strcpy(char *dst, char *src)
{
    size_t i = 0;
    while ((dst[i] = src[i]) != '\0') i++;
}

int main(void)
{
    char t[] = "hello, world";
    char buf[32];
    my_strcpy(buf, t);
    printf("%zu %zu [%s]\n", my_strlen(t), strlen(t), buf);
    return 0;
}

```

`while ((dst[i] = src[i]) != '\0')` copies the terminator too, then stops — the *value* of an assignment is the value assigned, so the loop tests the character it just copied. The extra parentheses are required; without them the `!=` binds tighter and

you'd be assigning the comparison result.

Two things to notice. First, `'\0'` is just the integer 0, so `while (s[i])` works identically — write whichever reads better to you. Second, `my_strcpy` has exactly the same fatal flaw as the real `strcpy`: it has no idea how big `dst` is. That's Exercise 7.4.

In Chapter 11 you'll rewrite both of these by walking the pointer instead of indexing, and in Chapter 16 you'll add `const` to the parameters that are only read.

7.2

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#include <stdbool.h>

void to_upper(char *s)
{
    for (size_t i = 0; s[i]; i++)
        s[i] = toupper((unsigned char)s[i]);
}

void reverse_str(char *s)
{
    size_t n = strlen(s);
    for (size_t i = 0; i + 1 < n - i; i++) {
        char t = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = t;
    }
}

bool is_palindrome(char *s)
{
    size_t i = 0, j = strlen(s);

    while (i < j) {
        if (!isalpha((unsigned char)s[i])) { i++; continue; }
        if (!isalpha((unsigned char)s[j - 1])) { j--; continue; }
        if (tolower((unsigned char)s[i]) != tolower((unsigned char)s[j - 1]))
            return false;
        i++;
        j--;
    }
    return true;
}

int main(void)
{
    char s[] = "A man, a plan, a canal: Panama";
    printf("%s\n", is_palindrome(s) ? "palindrome" : "not");
    to_upper(s);
    reverse_str(s);
    printf("%s\n", s);
    return 0;
}
```

`is_palindrome` uses `j` as a one-past-the-end index, which is why it reads `s[j - 1]`. That avoids the underflow you'd get from `size_t j = strlen(s) - 1` on an empty string, where `0 - 1` wraps to a huge number rather than going negative.

[ahead: Ch14–15] **The `(unsigned char)` cast on every `ctype.h` call is not optional, and it is not decoration.** These

functions take an `int` whose value must be representable as `unsigned char` or equal `E0F`. On platforms where plain `char` is signed, a byte above 127 becomes a negative number and you get undefined behaviour — a real crash on some libc implementations, not a theoretical one. Every serious C codebase writes this cast; most tutorials don't. Chapters 14 and 15 explain signedness and casts properly. Until then, copy it.

7.3

```
#include <stdio.h>
#include <ctype.h>
#include <stdbool.h>

int main(void)
{
    char s[] = " The quick brown fox jumps over the lazy dog. ";
    int words = 0, letters = 0, vowels = 0;
    bool in_word = false;

    for (size_t i = 0; s[i]; i++) {
        unsigned char c = s[i];

        if (isspace(c)) {
            in_word = false;
        } else if (!in_word) {
            words++;
            in_word = true;
        }

        if (isalpha(c)) {
            letters++;
            switch (tolower(c)) {
                case 'a': case 'e': case 'i': case 'o': case 'u':
                    vowels++;
                    break;
            }
        }
    }

    printf("words %d, letters %d, vowels %d\n", words, letters, vowels);
    return 0;
}
```

The `in_word` flag is the idea worth keeping: you count *transitions into* a word, not spaces. Counting spaces breaks on doubled spaces and on leading or trailing whitespace, both of which this test string has on purpose.

[ahead: Ch14] Copying each character into an `unsigned char c` up front does the same job as casting at every `ctype.h` call, and reads better when you use the character several times.

7.4

```

#include <stdio.h>
#include <string.h>

int main(void)
{
    char src[] = "this string is far too long";
    char buf[8];

    /* The fix you can write with Chapter 7 material: check first. */
    if (strlen(src) + 1 <= sizeof buf) {
        strcpy(buf, src);
        printf("[%s]\n", buf);
    } else {
        printf("refused: need %zu bytes, have %zu\n", strlen(src) + 1, sizeof buf);
    }
    return 0;
}

```

The `+ 1` is for the terminator, and it's where off-by-one bugs live. `sizeof buf` is 8, so a 7-character string is the longest that fits.

ASan on the unguarded version reports `stack-buffer-overflow`, names the exact byte offset past the object, and prints a memory map showing the 8-byte allocation. Read the whole report — learning to parse it is worth more than this exercise.

The fix you'll actually use in real code is `snprintf`, which arrives in Chapter 9. It bounds the write *and* always terminates:

```

snprintf(buf, sizeof buf, "%s", src); /* truncates safely, always NUL-terminated */

```

Note that GCC may warn on the guarded `strcpy` anyway (`-Wstringop-overflow`): it can see both the literal and the buffer size at compile time and doesn't track your guard. That's the optimizer being helpful, not a mistake in your code — a useful reminder that a warning is a heuristic to read, not an order to obey.

Why `strcpy` is not the fix: if the source is longer than `n`, `strcpy` copies `n` bytes and does **not** write a terminator. You get an unterminated buffer, and the next `strlen` or `printf("%s")` runs off the end — you've converted an obvious crash into a subtle one. It also pads with NULs to the full `n` when the source is shorter, which is wasted work. `strcpy` was designed for fixed-width record fields in the 1970s, not for safe string copying. Use `snprintf`.

Chapter 8 — Structs

8.1

```

#include <stdio.h>
#include <math.h>

struct point { double x, y; };
struct rect { struct point tl, br; };

double dist(struct point a, struct point b)
{
    double dx = a.x - b.x, dy = a.y - b.y;
    return sqrt(dx * dx + dy * dy);
}

double rect_area(struct rect r)
{
    return fabs(r.br.x - r.tl.x) * fabs(r.tl.y - r.br.y);
}

double rect_perimeter(struct rect r)
{
    return 2 * (fabs(r.br.x - r.tl.x) + fabs(r.tl.y - r.br.y));
}

int main(void)
{
    struct rect r = {{0, 10}, {4, 2}};
    printf("area %.2f perimeter %.2f diag %.2f\n",
           rect_area(r), rect_perimeter(r), dist(r.tl, r.br));
    return 0;
}

```

Compile with `-lm`.

8.2

```

#include <stdio.h>

struct point { double x, y; };
struct rect { struct point tl, br; };

void scale_by_value(struct rect r, double f)
{
    r.br.x *= f;
    r.br.y *= f;    /* modifies a copy; caller unaffected */
}

void scale_by_ptr(struct rect *r, double f)
{
    r->br.x *= f;
    r->br.y *= f;
}

int main(void)
{
    struct rect r = {{0, 0}, {2, 3}};
    scale_by_value(r, 10);
    printf("after by-value: %.1f %.1f\n", r.br.x, r.br.y);
    scale_by_ptr(&r, 10);
    printf("after by-ptr:   %.1f %.1f\n", r.br.x, r.br.y);
    printf("sizeof(struct rect) = %zu\n", sizeof(struct rect));
    return 0;
}

```

`sizeof(struct rect)` is 32 — four doubles. Passing by value copies all 32 bytes on every call. For a struct this small it's fine; for anything with an embedded array it's a real cost, which is why C code overwhelmingly passes `struct *` (or `const struct *` when it doesn't need to modify).

8.3

```

#include <stdio.h>
#include <string.h>

struct student {
    char name[32];
    int id;
    double grades[5];
};

double average(struct student *s)
{
    double sum = 0;
    for (int i = 0; i < 5; i++) sum += s->grades[i];
    return sum / 5;
}

int main(void)
{
    struct student cls[4] = {
        {"Ada Lovelace", 1001, {91, 88, 95, 79, 100}},
        {"Alan Turing", 1002, {85, 92, 78, 88, 90}},
        {"Grace Hopper", 1003, {99, 94, 91, 97, 93}},
        {"Dennis Ritchie", 1004, {77, 84, 90, 82, 88}},
    };

    /* selection sort by average, descending */
    for (int i = 0; i < 3; i++) {
        int best = i;
        for (int j = i + 1; j < 4; j++)
            if (average(&cls[j]) > average(&cls[best])) best = j;
        if (best != i) {
            struct student t = cls[i];
            cls[i] = cls[best];
            cls[best] = t;
        }
    }

    printf("%-18s %6s %8s\n", "NAME", "ID", "AVG");
    for (int i = 0; i < 4; i++)
        printf("%-18s %6d %8.2f\n", cls[i].name, cls[i].id, average(&cls[i]));
    return 0;
}

```

`struct student t = cls[i]; cls[i] = cls[best];` — structs assign by value in C, so this swaps whole records including the name array. You cannot do that with a bare array, which is one of the few places a struct wrapper buys you real convenience.

Chapter 9 — File Input/Output

9.1

```

#include <stdio.h>

int main(void)
{
    FILE *f = fopen("input.txt", "r");
    if (f == NULL) {
        perror("input.txt");
        return 1;
    }
    int c;
    while ((c = fgetc(f)) != EOF) putchar(c);
    fclose(f);
    return 0;
}

```

int c, not char c. fgetc returns an int so it can represent all 256 byte values *plus* the distinct sentinel EOF (which is -1). Store it in a char and on a signed-char platform a legitimate byte 0xFF compares equal to EOF, truncating your file early. This is one of the classic C bugs.

9.2

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main(void)
{
    FILE *f = fopen("input.txt", "r");
    if (!f) { perror("input.txt"); return 1; }

    char buf[64];
    int lineno = 1;
    bool at_line_start = true;

    while (fgets(buf, sizeof buf, f)) {
        if (at_line_start) printf("%4d ", lineno);
        fputs(buf, stdout);
        /* only a buffer ending in '\n' means we consumed a whole line */
        at_line_start = strchr(buf, '\n') != NULL;
        if (at_line_start) lineno++;
    }
    fclose(f);
    return 0;
}

```

The subtlety the exercise is testing: fgets reads *at most* size - 1 bytes, so a long line arrives in several calls. Incrementing lineno per call would number the fragments. Checking for the newline is what makes it correct.

9.3

```

#include <stdio.h>

int main(void)
{
    FILE *f = fopen("scores.txt", "w");
    if (!f) { perror("scores.txt"); return 1; }
    char *names[] = {"ada", "alan", "grace", "dennis", "ken",
                     "bjarne", "linus", "brian", "donald", "barbara"};
    for (int i = 0; i < 10; i++) fprintf(f, "%s %d\n", names[i], 50 + i * 7);
    fclose(f);

    f = fopen("scores.txt", "r");
    if (!f) { perror("scores.txt"); return 1; }

    char name[32], best_name[32] = "";
    int score, best = -1, n = 0;
    while (fscanf(f, "%31s %d", name, &score) == 2) {
        n++;
        if (score > best) { best = score; snprintf(best_name, sizeof best_name, "%s", name); }
    }
    fclose(f);
    printf("read %d records, best: %s %d\n", n, best_name, best);
    return 0;
}

```

`%31s` and not `%s`. A bare `%s` in `scanf` is an unbounded write into your buffer — the same defect as `gets()`, which was removed from the language in C11 for exactly this reason. The width must be one less than the buffer size to leave room for the NUL.

Checking `== 2` (the number of conversions requested) rather than `!= EOF` correctly stops on malformed input, not just end of file.

9.4

```

#include <stdio.h>
#include <string.h>

struct rec {
    int id;
    double value;
    char tag[16];
};

int main(void)
{
    struct rec out[5];
    for (int i = 0; i < 5; i++) {
        out[i].id = i;
        out[i].value = i * 1.5;
        snprintf(out[i].tag, sizeof out[i].tag, "tag%d", i);
    }

    FILE *f = fopen("recs.bin", "wb");
    if (!f) { perror("recs.bin"); return 1; }
    if (fwrite(out, sizeof(struct rec), 5, f) != 5) {
        fprintf(stderr, "short write\n");
        fclose(f);
        return 1;
    }
    fclose(f);

    struct rec in[5];
    f = fopen("recs.bin", "rb");
    if (!f) { perror("recs.bin"); return 1; }
    size_t got = fread(in, sizeof(struct rec), 5, f);
    fclose(f);

    printf("read %zu records\n", got);
    int ok = (got == 5) && (memcmp(out, in, sizeof out) == 0);
    printf("round trip %s\n", ok ? "OK" : "MISMATCH");
    return 0;
}

```

The "wb"/"rb" mode matters on Windows, where text mode translates `\n` to `\r\n` and would corrupt binary data. On Unix the `b` is a no-op but costs nothing.

Opened in a text editor you'll see the `tag` strings as readable text surrounded by binary garbage — the ints and doubles as raw bytes. Note that `memcmp` on whole structs works here only because both were fully initialized; padding bytes are indeterminate in general, which is why comparing structs with `memcmp` is usually a bug (see 20.2).

Chapter 10 — typedef

10.1

```

/* Form A: tag plus separate typedef – tag still available */
struct student { char name[32]; int id; };
typedef struct student Student;

/* Form B: combined, tag omitted – NOT self-referential-capable */
typedef struct { char name[32]; int id; } Student2;

/* Form C: combined, tag kept – this is the one to use for lists/trees */
typedef struct node {
    int val;
    struct node *next;    /* needs the tag; `Node *next` isn't visible yet */
} Node;

```

Form B cannot declare a member pointing at its own type, because the typedef name doesn't exist until the closing `}`. Keep the tag (Form C) and you get both. This is why nearly all real C code writes `typedef struct foo { ... } foo;`.

10.2

```

#include <stdio.h>

typedef char *string;
typedef int arr5[5];

void take(arr5 a)    /* silently becomes int *a */
{
    printf("inside function sizeof a = %zu\n", sizeof a);
}

int main(void)
{
    string a, b;      /* BOTH are char* – unlike `char *a, b;` */
    a = "hi";
    b = a;
    printf("%s %s %zu\n", a, b, sizeof a);

    arr5 nums = {1, 2, 3, 4, 5};
    printf("in main sizeof nums = %zu\n", sizeof nums);
    take(nums);
    return 0;
}

```

Two surprises worth internalizing:

1. `string a, b;` makes **both** pointers, whereas the equivalent `char *a, b;` makes `b` a plain `char`. The typedef fixed a real footgun — this is the one legitimate argument for pointer typedefs. The counterargument is that `string a` hides the fact that `a` is a pointer at all, which matters when someone has to decide whether to free it. Most style guides say don't hide pointers behind typedefs.
2. `arr5` as a parameter decays to `int *` exactly like a plain array does. The typedef does not preserve the size across a function call. `sizeof` gives 20 in `main` and 8 in the function.

Expected warning: GCC emits `-Wsizeof-array-argument` here, pointing straight at the bug. That warning firing is the whole point of the exercise — don't "fix" it.

Chapter 11 — Pointers II: Arithmetic

11.1

```

#include <stdio.h>

int main(void)
{
    int arr[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int *end = arr + sizeof arr / sizeof arr[0];
    int sum = 0;
    for (int *p = arr; p < end; p++) sum += *p;
    printf("%d\n", sum);
    return 0;
}

```

`arr + len` (one past the end) is explicitly legal to *form and compare against*, but not to dereference. That guarantee is what makes this idiom safe and it's why `end`-style iteration is everywhere in C and C++.

11.2

```

#include <stdio.h>
#include <stddef.h>

ptrdiff_t index_of(int *arr, int len, int target)
{
    for (int *p = arr; p < arr + len; p++)
        if (*p == target)
            return p - arr;    /* pointer difference IS the index */
    return -1;
}

int main(void)
{
    int a[] = {10, 20, 30, 40, 50};
    printf("%td\n", index_of(a, 5, 40));
    printf("%td\n", index_of(a, 5, 99));
    return 0;
}

```

Returning `ptrdiff_t` rather than `int` avoids a cast — the subtraction already has that type, so there's nothing to convert. Subtracting two pointers into the same array yields a `ptrdiff_t`, a *signed* type, printed with `%td`. Note it gives the difference in **elements**, not bytes: the compiler divides by `sizeof(int)` for you.

11.3

```

#include <stdio.h>
#include <string.h>

void memswap(void *a, void *b, size_t size)
{
    unsigned char *pa = a, *pb = b;
    for (size_t i = 0; i < size; i++) {
        unsigned char t = pa[i];
        pa[i] = pb[i];
        pb[i] = t;
    }
}

void reverse_generic(void *base, size_t n, size_t size)
{
    unsigned char *p = base;
    for (size_t i = 0, j = n ? n - 1 : 0; i < j; i++, j--)
        memswap(p + i * size, p + j * size, size);
}

struct pair { int a; double b; };

int main(void)
{
    int xs[] = {1, 2, 3, 4, 5};
    reverse_generic(xs, 5, sizeof xs[0]);
    for (int i = 0; i < 5; i++) printf("%d ", xs[i]);
    printf("\n");

    struct pair ps[] = {{1, 1.5}, {2, 2.5}, {3, 3.5}};
    reverse_generic(ps, 3, sizeof ps[0]);
    for (int i = 0; i < 3; i++) printf("(%d,%.1f) ", ps[i].a, ps[i].b);
    printf("\n");
    return 0;
}

```

[ahead: Ch14] `unsigned char *` is the correct type for byte-level work: it's the only type the standard lets you use to inspect the representation of any object, and it has no padding or trap representations. You cannot do arithmetic on `void *` in standard C (GCC allows it as an extension), so you must convert first.

Chapter 12 — Manual Memory Allocation

12.1

```

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t n;
    if (scanf("%zu", &n) != 1 || n == 0) {
        fprintf(stderr, "need a positive integer\n");
        return 1;
    }

    int *arr = malloc(n * sizeof *arr);
    if (arr == NULL) {
        perror("malloc");
        return 1;
    }

    for (size_t i = 0; i < n; i++) arr[i] = i * i;
    for (size_t i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");

    free(arr);
    return 0;
}

```

Reading straight into a `size_t` with `%zu` avoids converting a signed count into an unsigned one, which is the shape of a real class of allocation bug: a negative `int` becomes an enormous `size_t`. `sizeof *arr` rather than `sizeof(int)`: if you later change `arr` to a `long *`, the allocation follows automatically. Bugs where the type changed but the `sizeof` didn't are nasty and silent.

Casting `malloc`'s return is unnecessary in C (`void *` converts implicitly) and can hide a missing `#include <stdlib.h>`. Don't cast it.

12.2

```

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    size_t n = 8;
    int *m = malloc(n * sizeof *m);
    int *c = calloc(n, sizeof *c);
    if (!m || !c) { perror("alloc"); free(m); free(c); return 1; }

    printf("malloc:"); for (size_t i = 0; i < n; i++) printf(" %d", m[i]);
    printf("\ncalloc:"); for (size_t i = 0; i < n; i++) printf(" %d", c[i]);
    printf("\n");

    free(m);
    free(c);
    return 0;
}

```

`calloc` zeroes; `malloc` does not. But reading uninitialized `malloc` memory is **undefined behaviour**, not merely "unpredictable values" — ASan/MSan will flag it, and the compiler is entitled to assume it doesn't happen. So the `malloc` line above is technically a bug written on purpose to see the effect. In real code, never read before you write.

`calloc(n, size)` also checks `n * size` for overflow internally, which `malloc(n * size)` does not. That's a genuine security

argument for `calloc`.

12.3

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct vec {
    int *data;
    size_t len, cap;
};

bool vec_init(struct vec *v)
{
    v->data = NULL;
    v->len = v->cap = 0;
    return true;
}

bool vec_push(struct vec *v, int x)
{
    if (v->len == v->cap) {
        size_t newcap = v->cap ? v->cap * 2 : 8;
        int *tmp = realloc(v->data, newcap * sizeof *tmp);
        if (tmp == NULL) return false; /* v->data still valid! */
        v->data = tmp;
        v->cap = newcap;
    }
    v->data[v->len++] = x;
    return true;
}

int vec_get(struct vec *v, size_t i) { return v->data[i]; }

void vec_free(struct vec *v)
{
    free(v->data);
    v->data = NULL;
    v->len = v->cap = 0;
}

int main(void)
{
    struct vec v;
    vec_init(&v);
    for (int i = 0; i < 1000; i++)
        if (!vec_push(&v, i * 3)) { fprintf(stderr, "oom\n"); break; }

    printf("len %zu cap %zu first %d last %d\n",
           v.len, v.cap, vec_get(&v, 0), vec_get(&v, v.len - 1));
    vec_free(&v);
    return 0;
}
```

The single most important line here is `int *tmp = realloc(...)`. Writing `v->data = realloc(v->data, ...)` looks natural and leaks the entire old buffer whenever `realloc` fails and returns `NULL`. Always `realloc` into a temporary and only commit on success.

Doubling (rather than adding a constant) gives amortized $O(1)$ pushes. Growing by 1 each time makes 1000 pushes do 1000

reallocs and ~500,000 element copies.

`vec_get` ought to take a `const struct vec *` — it only reads. That qualifier is Chapter 16, and the Chapter 17 version of this file adds it.

12.4

```
#include <stdio.h>
#include <stdlib.h>

char *read_file(char *path, size_t *out_len)
{
    FILE *f = fopen(path, "rb");
    if (!f) return NULL;

    size_t cap = 4096, len = 0;
    char *buf = malloc(cap);
    if (!buf) { fclose(f); return NULL; }

    for (;;) {
        if (len == cap) {
            size_t newcap = cap * 2;
            char *tmp = realloc(buf, newcap);
            if (!tmp) { free(buf); fclose(f); return NULL; }
            buf = tmp;
            cap = newcap;
        }
        size_t got = fread(buf + len, 1, cap - len, f);
        len += got;
        if (got == 0) break;           /* EOF or error */
    }

    if (ferror(f)) { free(buf); fclose(f); return NULL; }
    fclose(f);

    char *tmp = realloc(buf, len + 1); /* shrink to fit, room for NUL */
    if (tmp) buf = tmp;
    buf[len] = '\0';

    if (out_len) *out_len = len;
    return buf;
}

int main(void)
{
    size_t len;
    char *text = read_file("input.txt", &len);
    if (!text) { perror("read_file"); return 1; }
    printf("%zu bytes, first 40: %.40s\n", len, text);
    free(text);
    return 0;
}
```

Distinguishing EOF from a read error requires `ferror` — `fread` returning a short count means either. An empty file gives `len == 0` and a valid one-byte buffer containing just the NUL, which is the right answer, not an error.

The shrink-to-fit `realloc` is allowed to fail, and if it does the original (larger) buffer is still valid — hence the `if (tmp)` rather than an unconditional assignment.

Chapter 13 — Scope

13.1

```
#include <stdio.h>

int count = 100;    /* file scope */

int main(void)
{
    int local = 10;
    printf("global %d, local %d\n", count, local);
    {
        int inner = 1;
        printf("global %d, local %d, inner %d\n", count, local, inner);
    }
    /* inner is gone here */
    return 0;
}
```

If you name all three `count`, the inner ones *hide* the outer ones and there is no syntax to reach past the shadow (unlike C++'s `::count`). `-Wshadow` reports “declaration of ‘count’ shadows a global declaration”. Turn `-Wshadow` on in real projects; shadowing bugs are silent and infuriating.

13.2

Chapter 13 is about *scope* — where a name is visible. The `static` storage class is Chapter 16, so here's the file-scope version:

```
#include <stdio.h>

int id_counter = 0;    /* file scope: visible to every function below */

int next_id(void)
{
    id_counter++;
    return id_counter;
}

void reset_ids(void)
{
    id_counter = 0;
}

int main(void)
{
    int a = next_id();
    int b = next_id();
    int c = next_id();
    printf("%d %d %d\n", a, b, c);

    reset_ids();
    printf("%d\n", next_id());
    return 0;
}
```

Calling `next_id()` three times into separate variables on separate lines, rather than three times inside one `printf`, is deliberate — see 3.3. Argument evaluation order is unspecified, so the inline version could legally print `3 2 1`.

Both functions can touch `id_counter` because file scope runs from the point of declaration to the end of the file. Declare it

below `next_id` instead and that function stops compiling, which is worth trying once.

Two things Chapter 16 adds to this picture:

- Marking the variable `static` gives it **internal linkage** — still visible to every function in this file, but invisible to the linker, so no other source file can reach it even with `extern`. That’s how you make a module-private variable.
- Moving it *inside* `next_id` as a `static` local keeps the value between calls while hiding it from `reset_ids` entirely.

Chapter 14 — Types II

14.1

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("char      %zu [%d, %d]\n", sizeof(char), CHAR_MIN, CHAR_MAX);
    printf("short     %zu [%d, %d]\n", sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("int        %zu [%d, %d]\n", sizeof(int), INT_MIN, INT_MAX);
    printf("long       %zu [%ld, %ld]\n", sizeof(long), LONG_MIN, LONG_MAX);
    printf("long long   %zu [%lld, %lld]\n", sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("unsigned    %zu [0, %u]\n", sizeof(unsigned), UINT_MAX);
    printf("size_t      %zu\n", sizeof(size_t));
    printf("float       %zu %d digits max %g\n", sizeof(float), FLT_DIG, FLT_MAX);
    printf("double      %zu %d digits max %g\n", sizeof(double), DBL_DIG, DBL_MAX);
    printf("long double %zu %d digits\n", sizeof(long double), LDBL_DIG);
    return 0;
}
```

`%zu` is the format specifier for `size_t`, which is what `sizeof` returns. Using `%d` there is a real (if usually harmless-looking) bug on 64-bit systems.

14.2

```
for (unsigned int i = 5; i >= 0; i--) { ... } /* INFINITE LOOP */
```

An unsigned value is never negative, so `i >= 0` is always true. When `i` is 0 and you decrement, it wraps to `UINT_MAX` (4294967295) and the loop runs forever. GCC with `-Wextra` warns: “comparison of unsigned expression `>= 0` is always true”.

Fixes: use a signed loop variable, or invert to `for (unsigned i = 5; i-- > 0; )` which is the standard “countdown including zero” idiom.

`(unsigned)-1` is `UINT_MAX`. Conversion to an unsigned type is defined as reducing modulo 2^N , so `-1` becomes $2^{32} - 1$. This is *defined behaviour*, unlike the signed overflow below — which is why `(unsigned)-1` is a legitimate portable way to write “all bits set”.

14.3

```

#include <stdio.h>
#include <limits.h>

int main(void)
{
    unsigned int u = UINT_MAX;
    u = u + 1;           /* DEFINED: wraps to 0 */
    printf("unsigned wrap: %u\n", u);

    volatile int i = INT_MAX; /* volatile stops constant folding */
    int j = i + 1;           /* UNDEFINED BEHAVIOUR */
    printf("signed overflow: %d\n", j);
    return 0;
}

```

Unsigned arithmetic is defined to wrap modulo 2^N . **Signed overflow is undefined behaviour** — the compiler is allowed to assume it never happens, and does: it will optimize `if (x + 1 < x)` to `false` outright. `-fsanitize=undefined` catches the signed case at runtime with “signed integer overflow: 2147483647 + 1 cannot be represented in type ‘int’” and says nothing about the unsigned one.

Chapter 15 — Types III: Conversions

15.1

```

#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <errno.h>
#include <ctype.h>
#include <stdbool.h>

bool parse_int(char *s, int *out)
{
    if (s == NULL || *s == '\0') return false;

    errno = 0;
    char *end;
    long v = strtol(s, &end, 10);

    if (end == s) return false;           /* no digits at all */
    while (isspace((unsigned char)*end)) end++; /* allow trailing space */
    if (*end != '\0') return false;       /* trailing garbage */
    if (errno == ERANGE || v < INT_MIN || v > INT_MAX) return false;

    *out = (int)v;
    return true;
}

int main(void)
{
    char *tests[] = {"42", "42abc", "", " 42", "999999999999999", "-7", "0x1A"};
    for (size_t i = 0; i < sizeof tests / sizeof tests[0]; i++) {
        int v;
        if (parse_int(tests[i], &v)) printf("[%s] -> %d\n", tests[i], v);
        else printf("[%s] -> REJECTED\n", tests[i]);
    }
    return 0;
}

```

" 42" is accepted because `strtol` skips *leading* whitespace itself. "0x1A" is rejected with base 10 (it stops at the `x`); pass base 0 and `strtol` auto-detects hex and octal prefixes.

You must set `errno = 0` before the call, because `strtol` only sets it on error and never clears it. And `strtol` returns `long`, so on a platform where `long` is wider than `int` you need the explicit range check on top of `ERANGE`.

These parameters should be `const char *` — the function only reads them. That's the next chapter.

This is why `atoi` is unusable: it cannot report *any* error. `atoi("abc")` returns 0, indistinguishable from `atoi("0")`.

15.2

```

5 / 2      == 2      /* both int: integer division truncates toward zero */
5.0 / 2    == 2.5    /* 2 promoted to double by usual arithmetic conversions */
5 / 2.0    == 2.5    /* same, other side */
(float)5 / 2 == 2.5  /* cast applies to 5 only; binds tighter than / */
(float)(5/2) == 2.0  /* division happens FIRST in int, then converts */
-7 / 2     == -3     /* C99+ truncates toward zero, not toward -inf */
-7 % 2     == -1     /* sign of % follows the dividend */

```

The `(float)(5/2)` case is the trap: the cast is too late, the precision is already gone. And `-7 % 2 == -1` catches people coming from Python, where it's `1`. If you need a always-positive modulus in C, write `((a % b) + b) % b`.

15.3

```

#include <stdio.h>
#include <ctype.h>

int digit_to_int(char c) { return c - '0'; }
char int_to_digit(int v) { return (char)('0' + v); }

int hex_to_int(char c)
{
    if (isdigit((unsigned char)c)) return c - '0';
    if (c >= 'a' && c <= 'f')      return c - 'a' + 10;
    if (c >= 'A' && c <= 'F')      return c - 'A' + 10;
    return -1;
}

int main(void)
{
    printf("%d %c %d %d %d\n",
        digit_to_int('7'), int_to_digit(3),
        hex_to_int('B'), hex_to_int('b'), hex_to_int('z'));
    return 0;
}

```

`c - '0'` is portable; `c - 48` is not. The standard guarantees the decimal digits `'0'` through `'9'` are contiguous and increasing — but it makes **no such guarantee for letters**, which is why the hex function tests ranges instead of assuming `'a' .. 'f'` are contiguous. (They are in ASCII; they are not in EBCDIC. In practice you'll never meet EBCDIC, but the habit costs nothing.)

`getchar()` returns `int` for the same reason `fgetc` does: it needs 256 distinct byte values plus a separate `E0F` sentinel, which doesn't fit in a `char`.

Chapter 16 — Types IV: Qualifiers and Specifiers

16.1

```

#include <stdio.h>

int main(void)
{
    char s1[] = "hello", s2[] = "world";

    const char *a = s1;          /* pointer to const char */
    a = s2;                      /* OK — pointer is mutable */
    /* a[0] = 'X'; */           /* ERROR — data is const */

    char * const b = s1;         /* const pointer to char */
    /* b = s2; */               /* ERROR — pointer is const */
    b[0] = 'X';                 /* OK — data is mutable */

    const char * const c = s1;  /* const pointer to const char */
    /* c = s2; */               /* ERROR */
    /* c[0] = 'X'; */           /* ERROR */

    printf("%s %s %s\n", a, b, c);
    return 0;
}

```

Read declarations **right to left** from the identifier: `const char *a` is “a is a pointer to a char that is const”; `char * const b` is

"b is a const pointer to a char". The `const` binds to whatever is immediately to its left, unless it's leftmost, in which case it binds right.

16.2

```
#include <stdio.h>

size_t my_strlen(const char *s)
{
    const char *p = s;
    while (*p) p++;
    return (size_t)(p - s);
}

int count_char(const char *s, char target)
{
    int n = 0;
    for (; *s; s++) if (*s == target) n++;
    return n;
}

int main(void)
{
    const char *msg = "mississippi";
    printf("%zu %d\n", my_strlen(msg), count_char(msg, 's'));
    return 0;
}
```

Writing `*s = 'x'` inside gives "assignment of read-only location". Note that `s++` is still fine — the *parameter* isn't const, only what it points to.

This is the promised upgrade to Exercise 7.1: same function, now with the read-only promise made explicit.

What `const` buys the caller: a compile-time guarantee that this function will not modify their buffer. It's part of the function's contract, checked by the compiler. It also lets callers pass string literals, which are not writable — without `const`, passing "hello" to your function is a warning and writing through it is undefined behaviour.

16.3

```
/* --- file1.c --- */
static int hidden = 42; /* internal linkage: invisible to the linker */
int shared = 99; /* external linkage */
int get_hidden(void) { return hidden; }

/* --- file2.c --- */
#include <stdio.h>
extern int shared;
/* extern int hidden; */ /* compiles, but fails at LINK time */
int get_hidden(void);

int main(void)
{
    printf("%d %d\n", shared, get_hidden());
    return 0;
}
```

Uncommenting the `hidden` `extern` gives you undefined reference to 'hidden' from the **linker** — the compiler is perfectly happy, because `extern` is a promise that the symbol exists somewhere. Nothing checks that promise until link time. Understanding which errors come from which tool is most of what makes multifile C tractable.

Chapter 17 — Multifile Projects

17.1

```
/* ===== vec.h ===== */
#ifndef VEC_H
#define VEC_H

#include <stddef.h>
#include <stdbool.h>

struct vec {
    int *data;
    size_t len, cap;
};

void vec_init(struct vec *v);
bool vec_push(struct vec *v, int x);
int vec_get(const struct vec *v, size_t i);
void vec_free(struct vec *v);

#endif
```

```
/* ===== vec.c ===== */
#include <stdlib.h>
#include "vec.h"

void vec_init(struct vec *v) { v->data = NULL; v->len = v->cap = 0; }

bool vec_push(struct vec *v, int x)
{
    if (v->len == v->cap) {
        size_t newcap = v->cap ? v->cap * 2 : 8;
        int *tmp = realloc(v->data, newcap * sizeof *tmp);
        if (!tmp) return false;
        v->data = tmp;
        v->cap = newcap;
    }
    v->data[v->len++] = x;
    return true;
}

int vec_get(const struct vec *v, size_t i) { return v->data[i]; }

void vec_free(struct vec *v) { free(v->data); v->data = NULL; v->len = v->cap = 0; }
```

```

/* ===== main.c ===== */
#include <stdio.h>
#include "vec.h"

int main(void)
{
    struct vec v;
    vec_init(&v);
    for (int i = 0; i < 100; i++) vec_push(&v, i);
    printf("%zu %d\n", v.len, vec_get(&v, 99));
    vec_free(&v);
    return 0;
}

```

```

CC      = gcc
CFLAGS  = -std=c17 -Wall -Wextra -g -fsanitize=address,undefined
OBJ      = main.o vec.o

prog: $(OBJ)
    $(CC) $(CFLAGS) -o $@ $(OBJ)

main.o: main.c vec.h
vec.o:  vec.c vec.h

clean:
    rm -f prog $(OBJ)

```

Listing `vec.h` as a prerequisite is what makes `make` rebuild both `.o` files when the header changes. Without it you get stale objects and baffling crashes — the classic symptom is a struct whose layout differs between two translation units.

17.2

Break	Error and from where
No include guard, header included twice	Compiler: “redefinition of ‘struct vec’”
Function <i>defined</i> in the header, two <code>.c</code> files include it	Linker: “multiple definition of ‘vec_get’”
Global variable defined (not <code>extern</code>) in the header	Linker: “multiple definition of ‘thing’”

The pattern: the compiler sees one translation unit at a time and catches duplicates *within* it. The linker sees all the object files and catches duplicates *across* them. So header hygiene errors mostly surface at link time, with messages that don’t mention the header at all.

Fix for the last two: put only *declarations* in headers (`int vec_get(...);` and `extern int thing;`), and *definitions* in exactly one `.c`. The modern alternative for small functions is `static inline` in the header, which gives each TU its own copy with no linker conflict.

Chapter 18 — The Outside Environment

18.1

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);
    printf("argv[argc] is %s\n", argv[argc] == NULL ? "NULL" : "NOT NULL");
    return 0;
}
```

`argv[0]` is the program name as invoked, and `argc` is always at least 1 in practice (though the standard permits 0). The NUL terminator on the array is guaranteed by the standard, which is why you can iterate `argv` without `argc` if you prefer.

18.2

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>
#include <errno.h>
#include <stdbool.h>

static bool parse_int(char *s, int *out)
{
    if (!s || !*s) return false;
    errno = 0;
    char *end;
    long v = strtol(s, &end, 10);
    if (end == s || *end != '\0') return false;
    if (errno == ERANGE || v < INT_MIN || v > INT_MAX) return false;
    *out = (int)v;
    return true;
}

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr, "usage: %s NUM OP NUM (op: + - x /)\n", argv[0]);
        return EXIT_FAILURE;
    }

    int a, b;
    if (!parse_int(argv[1], &a) || !parse_int(argv[3], &b)) {
        fprintf(stderr, "%s: bad number\n", argv[0]);
        return EXIT_FAILURE;
    }

    if (strlen(argv[2]) != 1) {
        fprintf(stderr, "%s: bad operator\n", argv[0]);
        return EXIT_FAILURE;
    }

    switch (argv[2][0]) {
        case '+': printf("%d\n", a + b); break;
        case '-': printf("%d\n", a - b); break;
        case 'x': printf("%d\n", a * b); break;
        case '/':
            if (b == 0) { fprintf(stderr, "%s: divide by zero\n", argv[0]); return EXIT_FAILURE; }
            printf("%d\n", a / b);
            break;
        default:
            fprintf(stderr, "%s: unknown operator '%s'\n", argv[0], argv[2]);
            return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}

```

x instead of * because the shell expands * to the filenames in the current directory before your program ever sees it. Diagnostics go to **stderr**, results to **stdout** — that's what makes `prog 6 x 7 > out.txt` put only the answer in the file. Prefixing errors with `argv[0]` is the Unix convention and makes errors traceable in a pipeline.

18.3

```

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    const char *path = getenv("PATH");
    const char *home = getenv("HOME");
    const char *dbg = getenv("MYAPP_DEBUG");

    printf("PATH = %s\n", path ? path : "(not set)");
    printf("HOME = %s\n", home ? home : "(not set)");

    if (dbg != NULL)
        fprintf(stderr, "[debug] MYAPP_DEBUG=%s, debugging enabled\n", dbg);

    return 0;
}

```

`getenv` returns `NULL` for an unset variable — always check. The returned string belongs to the environment; don't free it and don't modify it. Test with `MYAPP_DEBUG=1 ./prog`.

Chapter 19 — The C Preprocessor

19.1

```

#define SQUARE_BAD(x)  x * x
#define SQUARE(x)      ((x) * (x))

SQUARE_BAD(2 + 3)      /* expands to 2 + 3 * 2 + 3 == 11, not 25 */
100 / SQUARE_BAD(5)    /* expands to 100 / 5 * 5 == 100, not 4 */

```

Two separate failures. The first is missing parens around each *argument*: substitution is textual, so `2 + 3 * 2 + 3` gets normal precedence. The second is missing parens around the *whole expansion*, so the surrounding `/` binds to the first factor only.

`((x) * (x))` fixes both. It still evaluates `x` twice, so `SQUARE(i++)` is broken beyond repair by parentheses — that's the argument for `static inline` functions over macros wherever you have the choice.

19.2

```

#include <stdio.h>

#ifdef NDEBUG
#define DBG(...) ((void)0)
#else
#define DBG(...) do { \
    fprintf(stderr, "[%s:%d %s] ", __FILE__, __LINE__, __func__); \
    fprintf(stderr, __VA_ARGS__); \
    fprintf(stderr, "\n"); \
} while (0)
#endif

int compute(int x)
{
    DBG("called with x=%d", x);
    return x * 2;
}

int main(void)
{
    DBG("starting");
    printf("%d\n", compute(21));
    return 0;
}

```

`__func__` is not a macro — it's a predefined identifier (an actual `static const char[]`), which is why it isn't spelled with the double underscores on both sides like `__FILE__` and `__LINE__`.

`((void)0)` rather than nothing for the release build: it keeps `DBG(...)` a valid expression statement, so `if (x) DBG("hi"); else ...` doesn't silently break when you define `NDEBUG`.

19.3

```

#include <stdio.h>

#define ARRAY_LEN(a) (sizeof (a) / sizeof (a)[0])

#define SWAP(a, b, type) do { \
    type swap_tmp_ = (a); \
    (a) = (b); \
    (b) = swap_tmp_; \
} while (0)

void broken(int arr[])
{
    printf("inside function ARRAY_LEN = %zu (WRONG)\n", ARRAY_LEN(arr));
}

int main(void)
{
    int a[10];
    printf("in main ARRAY_LEN = %zu\n", ARRAY_LEN(a));
    broken(a);

    int x = 1, y = 2;
    if (x < y) SWAP(x, y, int); /* works because of do-while(0) */
    printf("%d %d\n", x, y);
    return 0;
}

```

The `do { } while (0)` idiom: with plain braces, `if (c) SWAP(x,y,int); else foo();` expands to `if (c) { ... }; else foo();` — the stray semicolon after the block ends the `if`, and `else` becomes a syntax error. Wrapping in `do{}while(0)` makes the macro a single statement that *requires* the trailing semicolon, so it composes correctly everywhere.

Where `ARRAY_LEN` lies: inside `broken`, `arr` is a pointer, not an array. `sizeof(arr)` is 8 and `sizeof(arr[0])` is 4, so you get 2. Modern GCC does catch this specific shape with `-Wsizeof-array-argument` — that warning in your build output is expected here — but it cannot catch the general case, where the pointer arrived from a `malloc` several functions away. This is exactly why you always pass a length alongside an array parameter.

Chapter 20 — structs II

20.1

```
#include <stdio.h>

struct inner { int a, b, c; };
struct outer { struct inner in; char tag[8]; double val; };

int main(void)
{
    struct outer o = { .in = { .b = 5 }, .val = 2.5 };
    struct inner arr[4] = { [2] = { .a = 9 }, [0] = { .c = 1 } };

    printf("in: %d %d %d  tag[%s]  val %.1f\n", o.in.a, o.in.b, o.in.c, o.tag, o.val);
    for (int i = 0; i < 4; i++)
        printf("arr[%d] = %d %d %d\n", i, arr[i].a, arr[i].b, arr[i].c);
    return 0;
}
```

Any member you omit from a designated initializer is initialized as if it had static storage duration: zero for arithmetic types, NULL for pointers. That guarantee is why `struct thing t = {0};` is the idiomatic “zero everything” — and why it works even when the first member isn’t an int. In C23 you can write `= {}`.

Designators also let you initialize out of order (`[2]` before `[0]` above), which is legal and occasionally readable.

20.2

```
#include <stdio.h>
#include <stddef.h>

struct a { char c; int i; char d; };
struct b { int i; char c, d; };

int main(void)
{
    printf("struct a: size %zu\n", sizeof(struct a));
    printf("  c at %zu, i at %zu, d at %zu\n",
        offsetof(struct a, c), offsetof(struct a, i), offsetof(struct a, d));
    printf("struct b: size %zu\n", sizeof(struct b));
    printf("  i at %zu, c at %zu, d at %zu\n",
        offsetof(struct b, i), offsetof(struct b, c), offsetof(struct b, d));
    return 0;
}
```

Typical x86-64 output: `struct a` is 12 bytes (c at 0, three padding bytes, i at 4, d at 8, three more trailing padding bytes); `struct b` is 8 bytes (i at 0, c at 4, d at 5, two trailing).

`int` must be 4-byte aligned, so the compiler inserts padding to make `i` land on a multiple of 4. Trailing padding exists so that

in an *array* of these structs, every element stays aligned.

Practical rule: declare members in descending order of alignment requirement (pointers and 8-byte types, then 4-byte, then 2, then chars). Reordering `struct a` to `{int i; char c, d;}` cuts it by a third for free.

This is also why `memcmp` on two structs is unreliable: the padding bytes are indeterminate, so two structs with identical member values can compare unequal.

20.3

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct node {
    int val;
    struct node *next;
};

/* Functions that can change the first node return the new head. */

struct node *push_front(struct node *head, int val)
{
    struct node *n = malloc(sizeof *n);
    if (!n) return head;          /* allocation failed: list unchanged */
    n->val = val;
    n->next = head;
    return n;
}

struct node *push_back(struct node *head, int val)
{
    struct node *n = malloc(sizeof *n);
    if (!n) return head;
    n->val = val;
    n->next = NULL;

    if (head == NULL) return n;   /* empty list: the new node IS the head */

    struct node *cur = head;
    while (cur->next != NULL) cur = cur->next;
    cur->next = n;
    return head;
}

struct node *find(struct node *head, int val)
{
    while (head != NULL) {
        if (head->val == val) return head;
        head = head->next;
    }
    return NULL;
}

struct node *delete_val(struct node *head, int val)
{
    if (head == NULL) return NULL;

    if (head->val == val) {        /* deleting the head is its own case */
        struct node *rest = head->next;
```

```

        free(head);
        return rest;
    }

    struct node *cur = head;
    while (cur->next != NULL) {
        if (cur->next->val == val) {
            struct node *dead = cur->next;
            cur->next = dead->next;
            free(dead);
            return head;
        }
        cur = cur->next;
    }
    return head; /* not found */
}

void print_list(struct node *head)
{
    while (head != NULL) {
        printf("%d -> ", head->val);
        head = head->next;
    }
    printf("NULL\n");
}

struct node *free_list(struct node *head)
{
    while (head != NULL) {
        struct node *next = head->next; /* save BEFORE freeing */
        free(head);
        head = next;
    }
    return NULL;
}

int main(void)
{
    struct node *head = NULL;
    for (int i = 3; i >= 1; i--) head = push_front(head, i);
    head = push_back(head, 4);
    head = push_back(head, 5);
    print_list(head);

    head = delete_val(head, 1); /* head case */
    head = delete_val(head, 5); /* tail case */
    head = delete_val(head, 3); /* middle case */
    print_list(head);
    printf("find(2) = %s\n", find(head, 2) ? "yes" : "no");

    head = free_list(head);
    return 0;
}

```

The pattern here — `head = push_front(head, x);` — is how you write a list using only what Chapter 20 has given you. Every function that might change which node is first hands the new first node back, and the caller reassigns. Forget one of those assignments and you leak the list or lose the new node; that's the cost of this approach.

Notice that `delete_val` needs a **separate branch for the head**, because there's no earlier node whose `next` field you could rewrite. Chapter 23 shows the pointer-to-pointer technique that makes that special case disappear entirely, and Exercise

23.2 has you come back and rewrite this.

`struct node *next = head->next;` before the `free` in `free_list` is mandatory. Reading `head->next` after freeing `head` is use-after-free — it will usually appear to work, and ASan will catch it immediately.

20.4

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define VAL_INT    0
#define VAL_DOUBLE 1
#define VAL_STRING 2

struct value {
    int type;
    union {
        int    i;
        double d;
        char   *s;
    } as;
};

struct value make_int(int i)
{
    struct value v;
    v.type = VAL_INT;
    v.as.i = i;
    return v;
}

struct value make_double(double d)
{
    struct value v;
    v.type = VAL_DOUBLE;
    v.as.d = d;
    return v;
}

struct value make_string(const char *s)
{
    struct value v;
    v.type = VAL_STRING;
    v.as.s = NULL;

    size_t n = strlen(s) + 1;
    v.as.s = malloc(n);
    if (v.as.s) memcpy(v.as.s, s, n);
    return v;
}

void value_print(const struct value *v)
{
    switch (v->type) {
        case VAL_INT:    printf("int(%d)\n", v->as.i);    break;
        case VAL_DOUBLE: printf("double(%g)\n", v->as.d); break;
        case VAL_STRING: printf("string(\"%s\")\n", v->as.s ? v->as.s : ""); break;
        default:         printf("unknown tag %d\n", v->type); break;
    }
}
```

```

}

void value_free(struct value *v)
{
    if (v->type == VAL_STRING) {
        free(v->as.s);
        v->as.s = NULL;
    }
}

int main(void)
{
    struct value vals[] = { make_int(42), make_double(3.14), make_string("hello") };
    size_t n = sizeof vals / sizeof vals[0];

    printf("sizeof(struct value) = %zu\n", sizeof(struct value));
    for (size_t i = 0; i < n; i++) value_print(&vals[i]);
    for (size_t i = 0; i < n; i++) value_free(&vals[i]);
    return 0;
}

```

These constructors return a whole `struct` by value, which Chapter 8 covered. Chapter 32 introduces compound literals, which let you collapse each of these to a single `return` statement.

A union is only as large as its biggest member, so the whole `struct value` is 16 bytes rather than the 20 it would take as three separate fields. The tag is what makes it safe: **reading a union member other than the one last written is undefined behaviour** (with a narrow exception for common initial sequences), so the tag isn't bookkeeping, it's a correctness requirement.

The tags here are #defined integers because `enum` is Chapter 22. When you get there, come back and convert them — it's a strict upgrade, and not only for readability: a `switch` over an `enum` with no `default` label makes the compiler warn "enumeration value 'VAL_X' not handled in switch" the moment you add a fourth type. With plain `int` tags you get no such help, which is why this version needs the `default` case that the `enum` version deliberately omits. That warning is one of the best free bug-catchers in C.

Chapter 21 — Characters and Strings II

21.1

```

#include <stdio.h>

int main(void)
{
    printf("\n newline      %d\n", '\n');
    printf("\t tab          %d\n", '\t');
    printf("\r carriage ret %d\n", '\r');
    printf("\\ backslash    %d\n", '\\');
    printf("\" double quote %d\n", '\"');
    printf("\0 NUL          %d\n", '\0');
    printf("\a bell         %d\n", '\a');
    printf("\x41 hex escape  %d (%c)\n", '\x41', '\x41');
    printf("\101 octal escape %d (%c)\n", '\101', '\101');

    printf("literal: C:\\path\\to\\file and a \"quoted\" word\n");
    return 0;
}

```

In the *format string* every backslash has to be doubled, because the compiler processes escapes before `printf` ever sees the string. `'\0'` is just the integer 0; the “NUL character” and “the number zero” are the same thing in C, which is why `while (*p)` works as a string-end test.

21.2

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void hexdump(const void *data, size_t len)
{
    const unsigned char *p = data;
    for (size_t i = 0; i < len; i += 16) {
        printf("%08zx ", i);

        for (size_t j = 0; j < 16; j++) {
            if (i + j < len) printf("%02x ", p[i + j]);
            else             printf("   ");
            if (j == 7) printf(" ");
        }

        printf(" |");
        for (size_t j = 0; j < 16 && i + j < len; j++)
            putchar(isprint(p[i + j]) ? p[i + j] : '.');
        printf("|\\n");
    }
}

int main(void)
{
    const char *s = "Hello\\tworld!\\nCaf\\xc3\\xa9 \\xe2\\x98\\x83 done.";
    hexdump(s, strlen(s));
    return 0;
}
```

`const unsigned char *` again — the only correct type for looking at raw bytes. The `é` shows up as the two bytes `c3 a9` and the snowman as `e2 98 83`: that’s UTF-8, and it’s the concrete demonstration for Exercise 27.1 that `strlen` counts bytes, not characters.

Chapter 22 — enum

22.1

```

#include <stdio.h>

enum day { SUN, MON, TUE, WED, THU, FRI, SAT, DAY_COUNT };
enum direction { NORTH = 1, EAST = 2, SOUTH = 4, WEST = 8 };

const char *day_name_switch(enum day d)
{
    switch (d) {
        case SUN: return "Sunday";
        case MON: return "Monday";
        case TUE: return "Tuesday";
        case WED: return "Wednesday";
        case THU: return "Thursday";
        case FRI: return "Friday";
        case SAT: return "Saturday";
        case DAY_COUNT: break;
    }
    return "?";
}

static const char *day_table[] = {
    "Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"
};

const char *day_name_table(enum day d)
{
    return (d >= SUN && d < DAY_COUNT) ? day_table[d] : "?";
}

int main(void)
{
    for (enum day d = SUN; d < DAY_COUNT; d++)
        printf("%d %s %s\n", d, day_name_switch(d), day_name_table(d));
    printf("SOUTH|WEST = %d\n", SOUTH | WEST);
    return 0;
}

```

The **switch version is the one that warns** when you add a day and forget to handle it (`-Wswitch`, on by default with `-Wall`). The table version silently returns garbage or `"?"`. That warning is worth the extra typing.

The `DAY_COUNT` trailing member is a standard trick for getting the number of enumerators without hardcoding it. Powers of two on `enum direction` make the values combinable as bit flags (Chapter 24).

22.2

```

#include <stdio.h>

enum light { RED, GREEN, YELLOW, LIGHT_COUNT };

const char *light_name(enum light l)
{
    switch (l) {
        case RED:    return "RED";
        case GREEN:  return "GREEN";
        case YELLOW: return "YELLOW";
        case LIGHT_COUNT: break;
    }
    return "INVALID";
}

enum light next_state(enum light l)
{
    switch (l) {
        case RED:    return GREEN;
        case GREEN:  return YELLOW;
        case YELLOW: return RED;
        case LIGHT_COUNT: break;
    }
    return RED; /* recover from an impossible state */
}

int main(void)
{
    enum light l = RED;
    for (int i = 0; i < 10; i++) {
        printf("%2d: %s\n", i, light_name(l));
        l = next_state(l);
    }
    printf("invalid: %s\n", light_name((enum light)99));
    return 0;
}

```

An `enum` variable is an integer type underneath, so nothing stops `(enum light)99` from existing. The compiler's exhaustiveness check covers the *named* values only — you still need a runtime fallback for data that came from a file, a network, or a cast.

Chapter 23 — Pointers III

23.1

```

#include <stdio.h>
#include <stdlib.h>

void alloc_and_fill(int **out, int n)
{
    int *p = malloc((size_t)n * sizeof *p);
    if (!p) { *out = NULL; return; }
    for (int i = 0; i < n; i++) p[i] = i * i;
    *out = p;
}

int main(void)
{
    int *arr = NULL;
    alloc_and_fill(&arr, 5);
    if (!arr) { perror("alloc"); return 1; }
    for (int i = 0; i < 5; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}

```

The reason for `int **`: to modify a caller's `int *`, you need that pointer's address — the same logic as `int *` to modify an `int`. Every “give me back an allocated thing” out-parameter in C is one level of indirection deeper than the thing itself.

23.2

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

char **make_names(const char *const *src, size_t n)
{
    char **names = malloc(n * sizeof *names);
    if (!names) return NULL;
    for (size_t i = 0; i < n; i++) {
        size_t len = strlen(src[i]) + 1;
        names[i] = malloc(len);
        if (!names[i]) { /* unwind on partial failure */
            for (size_t j = 0; j < i; j++) free(names[j]);
            free(names);
            return NULL;
        }
        memcpy(names[i], src[i], len);
    }
    return names;
}

void sort_names(char **names, size_t n)
{
    for (size_t i = 1; i < n; i++) { /* insertion sort on pointers */
        char *key = names[i];
        size_t j = i;
        while (j > 0 && strcmp(names[j - 1], key) > 0) {
            names[j] = names[j - 1];
            j--;
        }
        names[j] = key;
    }
}

void free_names(char **names, size_t n)
{
    for (size_t i = 0; i < n; i++) free(names[i]);
    free(names);
}

int main(void)
{
    const char *const src[] = {"grace", "dennis", "ada", "linus", "barbara"};
    size_t n = sizeof src / sizeof src[0];

    char **names = make_names(src, n);
    if (!names) { perror("alloc"); return 1; }

    sort_names(names, n);
    for (size_t i = 0; i < n; i++) printf("%s\n", names[i]);

    free_names(names, n);
    return 0;
}

```

Two things: the sort moves **pointers**, not string contents — one 8-byte copy per shift instead of copying whole strings, which is the entire reason `char **` layouts are common. And the partial-failure unwind in `make_names` is what separates toy code from correct code: if the fourth `malloc` fails you must free the first three or you've leaked.

23.3

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static void memswap(void *a, void *b, size_t size)
{
    unsigned char *pa = a, *pb = b;
    for (size_t i = 0; i < size; i++) {
        unsigned char t = pa[i];
        pa[i] = pb[i];
        pb[i] = t;
    }
}

void my_sort(void *base, size_t n, size_t size,
             int (*cmp)(const void *, const void *))
{
    unsigned char *p = base;
    for (size_t i = 1; i < n; i++)
        for (size_t j = i; j > 0 && cmp(p + (j - 1) * size, p + j * size) > 0; j--)
            memswap(p + (j - 1) * size, p + j * size, size);
}

int cmp_int_asc(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);      /* no overflow, unlike x - y */
}

int cmp_int_desc(const void *a, const void *b) { return cmp_int_asc(b, a); }

int cmp_str(const void *a, const void *b)
{
    return strcmp(*(const char *const *)a, *(const char *const *)b);
}

int main(void)
{
    int xs[] = {5, 3, 9, 1, 7};
    my_sort(xs, 5, sizeof xs[0], cmp_int_asc);
    for (int i = 0; i < 5; i++) printf("%d ", xs[i]);
    printf("\n");

    qsort(xs, 5, sizeof xs[0], cmp_int_desc);
    for (int i = 0; i < 5; i++) printf("%d ", xs[i]);
    printf("\n");

    const char *names[] = {"grace", "ada", "linus", "barbara"};
    my_sort(names, 4, sizeof names[0], cmp_str);
    for (int i = 0; i < 4; i++) printf("%s ", names[i]);
    printf("\n");
    return 0;
}
```

Two details that catch nearly everyone:

return x - y; is a bug. With `x = INT_MAX` and `y = -1` the subtraction overflows and gives a negative result — the comparator claims the larger value is smaller. `(x > y) - (x < y)` yields exactly -1, 0, or 1 with no arithmetic risk.

The string comparator takes `char **`, not `char *`. `qsort` hands your comparator the address of *an array element*, and each element is itself a `char *`. Forgetting one level of indirection here compiles fine and crashes at runtime. This is the single most common `qsort` mistake.

Chapter 24 — Bitwise Operations

24.1

```
#include <stdio.h>

void print_bits(unsigned int v)
{
    for (int i = 31; i >= 0; i--) {
        putchar((v >> i) & 1 ? '1' : '0');
        if (i % 4 == 0 && i) putchar(' ');
    }
    putchar('\n');
}

int main(void)
{
    unsigned int a = 0xF0F0u, b = 0x00FFu;
    printf("a      = "); print_bits(a);
    printf("b      = "); print_bits(b);
    printf("a & b   = "); print_bits(a & b);
    printf("a | b   = "); print_bits(a | b);
    printf("a ^ b   = "); print_bits(a ^ b);
    printf("~a    = "); print_bits(~a);
    printf("a << 4 = "); print_bits(a << 4);
    printf("a >> 4 = "); print_bits(a >> 4);
    return 0;
}
```

Do all bit twiddling on **unsigned** types. Right-shifting a negative signed value is implementation-defined (arithmetic vs logical shift), and left-shifting into or past the sign bit is undefined behaviour.

24.2

```

#include <stdio.h>
#include <stdbool.h>

unsigned set_bit(unsigned v, int n) { return v | (1u << n); }
unsigned clear_bit(unsigned v, int n) { return v & ~(1u << n); }
unsigned toggle_bit(unsigned v, int n) { return v ^ (1u << n); }
bool test_bit(unsigned v, int n) { return (v >> n) & 1u; }

int count_bits(unsigned v)
{
    int n = 0;
    while (v) { v &= v - 1; n++; } /* Kernighan: clears lowest set bit */
    return n;
}

bool is_power_of_two(unsigned v) { return v != 0 && (v & (v - 1)) == 0; }

int main(void)
{
    unsigned v = 0;
    v = set_bit(v, 0);
    v = set_bit(v, 3);
    v = set_bit(v, 7);
    printf("v = %u, bits set = %d, bit 3 = %d\n", v, count_bits(v), test_bit(v, 3));
    v = clear_bit(v, 3);
    v = toggle_bit(v, 1);
    printf("v = %u, bits set = %d\n", v, count_bits(v));
    for (unsigned x = 1; x <= 20; x++)
        if (is_power_of_two(x)) printf("%u ", x);
    printf("\n");
    return 0;
}

```

`1u << n` and not `1 << n`: with a plain `int` literal, `1 << 31` shifts into the sign bit, which is undefined behaviour.

`v &= v - 1` clears the lowest set bit, because subtracting 1 flips that bit to 0 and everything below it to 1; ANDing keeps only the higher bits. The loop therefore runs once per set bit rather than 32 times.

24.3

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

enum perm {
    PERM_READ   = 1u << 0,
    PERM_WRITE  = 1u << 1,
    PERM_EXEC   = 1u << 2,
    PERM_DELETE = 1u << 3,
};

unsigned perm_add(unsigned p, unsigned f) { return p | f; }
unsigned perm_remove(unsigned p, unsigned f) { return p & ~f; }
bool perm_has(unsigned p, unsigned f) { return (p & f) == f; }

/* --- bitset over a byte array --- */
#define BITSET_BYTES(n) (((n) + 7) / 8)

```

```

struct bitset { unsigned char *bytes; size_t nbits; };

bool bitset_init(struct bitset *bs, size_t nbits)
{
    bs->bytes = calloc(BITSET_BYTES(nbits), 1);
    bs->nbits = nbits;
    return bs->bytes != NULL;
}

void bitset_set(struct bitset *bs, size_t i) { bs->bytes[i / 8] |= (unsigned char)(1u << (i % 8)); }
void bitset_clear(struct bitset *bs, size_t i) { bs->bytes[i / 8] &= (unsigned char)~(1u << (i % 8)); }
bool bitset_test(const struct bitset *bs, size_t i) { return (bs->bytes[i / 8] >> (i % 8)) & 1u; }

size_t bitset_count(const struct bitset *bs)
{
    size_t n = 0;
    for (size_t b = 0; b < BITSET_BYTES(bs->nbits); b++) {
        unsigned char v = bs->bytes[b];
        while (v) { v &= (unsigned char)(v - 1); n++; }
    }
    return n;
}

void bitset_free(struct bitset *bs) { free(bs->bytes); bs->bytes = NULL; }

int main(void)
{
    unsigned p = 0;
    p = perm_add(p, PERM_READ | PERM_WRITE);
    printf("read? %d exec? %d rw? %d\n",
        perm_has(p, PERM_READ), perm_has(p, PERM_EXEC),
        perm_has(p, PERM_READ | PERM_WRITE));
    p = perm_remove(p, PERM_WRITE);
    printf("after remove, write? %d\n", perm_has(p, PERM_WRITE));

    struct bitset bs;
    if (!bitset_init(&bs, 1000)) return 1;
    for (size_t i = 0; i < 1000; i += 7) bitset_set(&bs, i);
    printf("bits set: %zu, bit 700: %d, bit 701: %d\n",
        bitset_count(&bs), bitset_test(&bs, 700), bitset_test(&bs, 701));
    bitset_clear(&bs, 700);
    printf("after clear, bit 700: %d, count %zu\n",
        bitset_test(&bs, 700), bitset_count(&bs));
    bitset_free(&bs);
    return 0;
}

```

`(p & f) == f` rather than `(p & f) != 0` — the first asks “has **all** of these flags”, the second “has **any**”. Both are useful; pick deliberately.

`(n + 7) / 8` is the integer ceiling-divide idiom. 1000 bits needs 125 bytes; 1001 bits needs 126. Getting this wrong by one byte is the classic bitset bug.

A bitset uses 1/8th the memory of a `bool` array (which is one byte per element) and 1/32nd of an `int` array. For the Chapter 6 sieve at scale, that’s the difference between fitting in cache and not.

Chapter 25 — Variadic Functions

25.1

```
#include <stdio.h>
#include <stdarg.h>

int sum_all(int count, ...)
{
    va_list ap;
    va_start(ap, count);
    int total = 0;
    for (int i = 0; i < count; i++) total += va_arg(ap, int);
    va_end(ap);
    return total;
}

double avg_all(int count, ...)
{
    if (count <= 0) return 0.0;
    va_list ap;
    va_start(ap, count);
    double total = 0;
    for (int i = 0; i < count; i++) total += va_arg(ap, double);
    va_end(ap);
    return total / count;
}

int main(void)
{
    printf("%d\n", sum_all(4, 10, 20, 30, 40));
    printf("%.2f\n", avg_all(3, 1.5, 2.5, 3.5));
    return 0;
}
```

There is **no way** for a variadic function to discover how many arguments it received, or their types. That information exists only at the call site and is thrown away. So every variadic API needs either an explicit count (this) or a sentinel value (`exec1` uses a trailing `NULL`) or a format string that describes the arguments (`printf`). If the caller lies, you get undefined behaviour with no diagnostic.

Also note `va_arg(ap, double)` — the default argument promotions apply to variadic arguments, so `float` becomes `double` and anything smaller than `int` becomes `int`. You cannot `va_arg(ap, float)`; it's undefined.

25.2

```

#include <stdio.h>
#include <stdarg.h>

void log_msg(const char *level, const char *fmt, ...)
{
    fprintf(stderr, "[%s] ", level);

    va_list ap;
    va_start(ap, fmt);
    vfprintf(stderr, fmt, ap);
    va_end(ap);

    fputc('\n', stderr);
}

int main(void)
{
    log_msg("INFO", "starting up, pid placeholder %d", 1234);
    log_msg("ERROR", "failed to open %s: code %d", "config.ini", 2);
    return 0;
}

```

The `v`-prefixed functions (`vprintf`, `vfprintf`, `vsnprintf`) exist precisely because you cannot forward `...` to another variadic function — you can only forward the `va_list`. Every logging library in C is built on this.

One caution: a `va_list` may be consumed by the call. If you need to use it twice (e.g. `vsnprintf` to measure, then again to write), you must `va_copy` it first.

Chapter 26 — Locale and Internationalization

26.1

```

#include <stdio.h>
#include <locale.h>

int main(void)
{
    printf("default (C locale): %f\n", 1234.5678);
    printf("locale is now: %s\n", setlocale(LC_ALL, NULL));

    const char *got = setlocale(LC_ALL, "");
    printf("after setlocale: %s\n", got ? got : "(failed)");
    printf("same number:      %f\n", 1234.5678);
    return 0;
}

```

`setlocale(LC_ALL, NULL)` *queries* without changing; `""` means “use the environment” (`LANG`, `LC_ALL`). A C program starts in the “C” locale regardless of the environment — you have to opt in. In a German locale the decimal separator becomes a comma, which silently breaks any code that parses its own output with `strtod`.

26.2

```

#include <stdio.h>
#include <locale.h>
#include <limits.h>

static void report(void)
{
    struct lconv *lc = localeconv();
    printf("  decimal_point   [%s]\n", lc->decimal_point);
    printf(" thousands_sep   [%s]\n", lc->thousands_sep);
    printf(" currency_symbol [%s]\n", lc->currency_symbol);
    printf(" int_curr_symbol [%s]\n", lc->int_curr_symbol);
    printf(" frac_digits      %d\n", lc->frac_digits == CHAR_MAX ? -1 : lc->frac_digits);
    printf(" p_cs_precedes    %d p_sign_posn %d\n", lc->p_cs_precedes, lc->p_sign_posn);
}

int main(void)
{
    printf("C locale:\n");
    report();
    setlocale(LC_ALL, "");
    printf("\nenvironment locale (%s):\n", setlocale(LC_ALL, NULL));
    report();
    return 0;
}

```

In the "C" locale most of these are empty strings and `frac_digits` is `CHAR_MAX`, the standard's "not available" marker.

Why hardcoding `.` is a bug: half the world writes 1.234,56 where the other half writes 1,234.56. Software that assumes one of them mangles currency amounts for the other. India uses a different *grouping* entirely (1,00,000), which is what `lc->grouping` describes.

Chapter 27 — Unicode, Wide Characters

27.1

```

#include <stdio.h>
#include <string.h>

size_t utf8_chars(const char *s)
{
    size_t n = 0;
    for (const unsigned char *p = (const unsigned char *)s; *p; p++)
        if ((*p & 0xC0) != 0x80) n++; /* not a continuation byte */
    return n;
}

int main(void)
{
    const char *s = "Café \xe2\x98\x83 na\xc3\xafve";
    printf("[%s]\n", s);
    printf("strlen (bytes)      = %zu\n", strlen(s));
    printf("utf8_chars (points) = %zu\n", utf8_chars(s));
    return 0;
}

```

A UTF-8 continuation byte always has its top two bits set to 10, so `(b & 0xC0) != 0x80` identifies the *first* byte of each character. `strlen` counts bytes and always will — the "é" is two bytes, the snowman is three, an emoji is often four.

Worth knowing: this counts **code points**, which still isn't "characters" as a user perceives them. A flag emoji is two code points; `é` can be written as one code point or as `e` plus a combining accent. Full correctness needs a library like ICU — the chapter says as much.

27.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <wchar.h>
#include <locale.h>

int main(void)
{
    setlocale(LC_ALL, "");          /* REQUIRED before any mb/wc conversion */

    const char *mb = "Café \xe2\x98\x83";
    size_t wlen = mbstowcs(NULL, mb, 0);
    if (wlen == (size_t)-1) { fprintf(stderr, "invalid multibyte\n"); return 1; }

    wchar_t *wide = malloc((wlen + 1) * sizeof *wide);
    if (!wide) return 1;
    mbstowcs(wide, mb, wlen + 1);

    printf("bytes: %zu, wide chars: %zu\n", strlen(mb), wlen);
    printf("wide:  %ls\n", wide);

    size_t blen = wcstombs(NULL, wide, 0);
    char *back = malloc(blen + 1);
    if (!back) { free(wide); return 1; }
    wcstombs(back, wide, blen + 1);
    printf("back:  %s\n", back);

    free(wide);
    free(back);
    return 0;
}
```

Passing `NULL` as the destination makes `mbstowcs` return the required length without writing anything — the standard two-pass sizing idiom.

Forget `setlocale` and the conversion runs in the "C" locale, where multibyte sequences aren't recognized: you get `(size_t)-1` or one wide char per byte. This is the number one cause of "my Unicode code doesn't work" in C.

Chapter 28 — Exiting a Program

28.1

```

#include <stdio.h>
#include <stdlib.h>

void first(void) { printf("handler first\n"); }
void second(void) { printf("handler second\n"); }
void third(void) { printf("handler third\n"); }

int main(void)
{
    atexit(first);
    atexit(second);
    atexit(third);
    printf("main done\n");
    exit(EXIT_SUCCESS);
}

```

Handlers run in **reverse order of registration** (LIFO): third, second, first. That's deliberate — it mirrors the order you'd tear down nested resources.

Way out	atexit handlers	stdio flushed
return from main	yes	yes
exit()	yes	yes
quick_exit()	no (at_quick_exit ones instead)	no
_Exit()	no	no
abort()	no	no

`_Exit` and `abort` skipping the flush is why a crashed program often loses its last few `printf` lines — they were still sitting in the buffer. `fflush(stdout)` before anything risky, or use `stderr`, which is unbuffered.

28.2

```

#include <stdio.h>
#include <assert.h>

int divide(int a, int b)
{
    assert(b != 0 && "divide: denominator must be non-zero");
    return a / b;
}

int main(void)
{
    printf("%d\n", divide(10, 2));
    printf("%d\n", divide(10, 0));    /* fires */
    return 0;
}

```

The `&& "message"` trick works because a string literal is always truthy, so it doesn't change the condition — but it *does* appear in the printed assertion text. Free documentation.

Never put a side effect in an `assert`. With `-DNDEBUG` the entire expression is removed by the preprocessor, so `assert(vec_push(&v, 1))` stops pushing anything in release builds. Asserts are for programmer errors (broken invariants, impossible states); use real error handling for things that can legitimately go wrong at runtime, like a failed `malloc` or bad user input.

Chapter 29 — Signal Handling

29.1

```
#include <stdio.h>
#include <signal.h>
#include <stdbool.h>

static volatile sig_atomic_t stop = 0;

static void on_sigint(int sig)
{
    (void)sig;
    stop = 1;
}

int main(void)
{
    if (signal(SIGINT, on_sigint) == SIG_ERR) {
        perror("signal");
        return 1;
    }

    printf("working... press Ctrl-C to stop\n");
    unsigned long i = 0;
    while (!stop) i++;

    printf("\ncaught SIGINT after %lu iterations, cleaning up\n", i);
    return 0;
}
```

`volatile sig_atomic_t` is the only type the standard promises you can safely share between a handler and the main flow. `volatile` stops the compiler from caching `stop` in a register and turning `while (!stop)` into an infinite loop — without it, an optimizing build may hang even though the handler runs.

29.2

```
static void unsafe_handler(int sig)
{
    (void)sig;
    printf("caught signal\n");    /* UNSAFE — do not do this */
}

static void safe_handler(int sig)
{
    (void)sig;
    stop = 1;                    /* the only thing you should generally do */
}
```

A signal can arrive at *any* instruction, including in the middle of a `printf` that’s already holding `stdio`’s internal buffer and lock. Calling `printf` again from the handler re-enters a function that isn’t reentrant: you can deadlock, corrupt the buffer, or double-free. It appears to work almost always, which makes it worse — it’s a bug that shows up under load, in production, once a month.

The standard permits a handler to do essentially three things: assign to a `volatile sig_atomic_t`, call `abort` / `_Exit` / `quick_exit`, or call `signal` for the same signal. Everything else is undefined. (POSIX adds a longer list of async-signal-safe functions, `write(2)` among them — which is why crash handlers use `write` and not `printf`.)

The chapter’s advice — “friends don’t let friends `signal()`” — is about the standard function’s underspecified semantics across platforms. On POSIX, use `sigaction`.

Chapter 30 — Variable-Length Arrays

30.1

```
#include <stdio.h>

int main(void)
{
    int n;
    if (scanf("%d", &n) != 1 || n <= 0 || n > 1000) {
        fprintf(stderr, "need 1..1000\n");
        return 1;
    }

    int arr[n];                /* VLA: size fixed at declaration time */
    for (int i = 0; i < n; i++) arr[i] = i * i;
    printf("sizeof(arr) = %zu (runtime!)\n", sizeof arr);
    printf("last = %d\n", arr[n - 1]);
    return 0;
}
```

The bound check matters. VLAs live on the **stack**, which is typically 8 MB, and there is no way to detect the failure — a too-large `n` blows the stack and your program dies with no diagnostic. That single fact is why the Linux kernel banned VLAs outright.

30.2

```
#include <stdio.h>

void print_matrix(int rows, int cols, int m[rows][cols])
{
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) printf("%4d", m[i][j]);
        printf("\n");
    }
}

int main(void)
{
    int a[2][3] = {{1,2,3},{4,5,6}};
    int b[3][2] = {{1,2},{3,4},{5,6}};
    print_matrix(2, 3, a);
    printf("\n");
    print_matrix(3, 2, b);
    return 0;
}
```

`rows` and `cols` must come **before** `m` in the parameter list — the compiler needs them in scope to describe `m`'s type. This is the real payoff of VLAs: without them, a function taking a 2D array must hardcode the column count, and you end up faking it with `int *m` plus manual `m[i * cols + j]` indexing.

Use `malloc` instead when the size is large, comes from untrusted input, or the array must outlive the function. VLAs are also optional in C11+ (`__STDC_NO_VLA__`), so portable code can't assume them.

Chapter 31 — goto

31.1

```
#include <stdio.h>

int main(void)
{
    int cube[3][3][3];
    int v = 0;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            for (int k = 0; k < 3; k++) cube[i][j][k] = v++;

    int target = 14, fi = -1, fj = -1, fk = -1;

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            for (int k = 0; k < 3; k++)
                if (cube[i][j][k] == target) {
                    fi = i; fj = j; fk = k;
                    goto found;
                }
found:
    if (fi >= 0) printf("found %d at [%d][%d][%d]\n", target, fi, fj, fk);
    else printf("not found\n");
    return 0;
}
```

The flag-variable alternative needs a `bool done` tested in all three loop conditions — three extra tests per iteration and three places to get wrong. `goto` out of nested loops is genuinely the clearer option, and it's one of the two uses essentially every C style guide permits.

31.2

```

#include <stdio.h>
#include <stdlib.h>

int process(const char *in_path, const char *out_path)
{
    int rc = -1;

    FILE *in = fopen(in_path, "r");
    if (!in) { perror(in_path); goto out; }

    FILE *out = fopen(out_path, "w");
    if (!out) { perror(out_path); goto close_in; }

    char *buf = malloc(4096);
    if (!buf) { perror("malloc"); goto close_out; }

    size_t n;
    while ((n = fread(buf, 1, 4096, in)) > 0)
        if (fwrite(buf, 1, n, out) != n) { perror("write"); goto free_buf; }

    rc = 0;

free_buf:
    free(buf);
close_out:
    fclose(out);
close_in:
    fclose(in);
out:
    return rc;
}

int main(void)
{
    return process("input.txt", "copy.txt") == 0 ? 0 : 1;
}

```

The cleanup labels appear in **reverse order of acquisition**, and each failure jumps to the label that releases exactly what was acquired *before* it. Every resource is freed on exactly one path, and adding a fourth resource means adding one label, not editing five `if` branches.

This is the dominant error-handling pattern in the Linux kernel and in most serious C libraries. The alternative — nested `if`s with duplicated cleanup in each branch — is where resource leaks actually come from.

Chapter 32 — Compound Literals and Generic Selections

32.1

```

#include <stdio.h>

struct point { int x, y; };

void show(struct point p) { printf("(%d, %d)\n", p.x, p.y); }
int sum(const int *a, int n)
{
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}

int main(void)
{
    show((struct point){ .x = 3, .y = 4 });           /* unnamed struct */
    printf("%d\n", sum((int[]){1, 2, 3, 4, 5}, 5)); /* unnamed array */

    struct point *p = &(struct point){ 7, 8 };       /* address of a literal */
    show(*p);
    return 0;
}

```

A compound literal at block scope has **automatic storage duration** — it lives until the end of the enclosing block, exactly like a local variable. So `p` above is valid for the rest of `main`, but returning `&(struct point){1,2}` from a function returns a dangling pointer. At file scope they get static duration instead.

32.2

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define TYPE_NAME(x) _Generic((x), \
    int:      "int",          \
    long:     "long",         \
    float:    "float",        \
    double:   "double",       \
    char:     "char",         \
    char *:   "char *",       \
    const char *: "const char *", \
    default:  "unknown")

#define my_abs(x) _Generic((x), \
    int:      abs,            \
    long:     labs,           \
    float:    fabsf,          \
    double:   fabs)(x)

int main(void)
{
    int i = -5;
    long l = -6L;
    double d = -7.5;
    char *s = "hi";

    printf("%s %s %s %s\n", TYPE_NAME(i), TYPE_NAME(l), TYPE_NAME(d), TYPE_NAME(s));
    printf("%d %ld %.1f\n", my_abs(i), my_abs(l), my_abs(d));
    return 0;
}

```

`_Generic` selects at **compile time** based on the static type of the controlling expression — there's no runtime dispatch and no cost. Note the trailing `(x)` in `my_abs`: the generic selection evaluates to a *function name*, which is then called. This is exactly how `<tgmath.h>` implements type-generic `sqrt`.

Compile with `-lm`.

Chapter 33 — Arrays Part II

33.1

```
#include <stdio.h>

int sum(int n, int arr[static n])
{
    int s = 0;
    for (int i = 0; i < n; i++) s += arr[i];
    return s;
}

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};
    printf("%d\n", sum(5, a));
    /* sum(10, a); */ /* GCC: "argument 2 of 'sum' must point to at least 10 elements" */
    return 0;
}
```

`int arr[static n]` is a *promise from the caller* that the pointer is non-NULL and points to at least `n` elements. It does not change the parameter's type — it's still `int *`. Two things it buys you: GCC and Clang can warn at compile time when they can see the caller passing something too small, and the optimizer may hoist loads knowing the memory is accessible.

33.2

```
#include <stdio.h>

int total(const int arr[const 10])
{
    int s = 0;
    for (int i = 0; i < 10; i++) s += arr[i];
    /* arr[0] = 1; */ /* ERROR: assignment of read-only location */
    /* arr = NULL; */ /* ERROR: assignment of read-only parameter 'arr' */
    return s;
}

int main(void)
{
    int a[10] = {1,2,3,4,5,6,7,8,9,10};
    printf("%d\n", total(a));
    return 0;
}
```

Both fail. The qualifier *inside* the brackets applies to the (implicit) pointer parameter itself, which is a syntax that exists nowhere else in C — `int arr[const 10]` means the same as `int * const arr`. The leading `const` applies to the elements, as usual.

Chapter 34 — setjmp, longjmp

34.1

```
#include <stdio.h>
#include <setjmp.h>

static jmp_buf env;

void level3(void) { printf(" level3, jumping\n"); longjmp(env, 42); }
void level2(void) { printf(" level2\n"); level3(); printf(" NEVER PRINTED\n"); }
void level1(void) { printf("level1\n"); level2(); }

int main(void)
{
    int rc = setjmp(env);
    if (rc == 0) {
        printf("setjmp returned 0 (first pass)\n");
        level1();
    } else {
        printf("setjmp returned %d (came back from longjmp)\n", rc);
    }
    return 0;
}
```

setjmp returns 0 the first time and the value passed to longjmp when jumped to. The frames for level1/level2/level3 are discarded without any cleanup — no free, no fclose, nothing.

34.2

```
#include <stdio.h>
#include <setjmp.h>

static jmp_buf exc_env;

#define TRY        if (setjmp(exc_env) == 0)
#define CATCH      else
#define THROW(code) longjmp(exc_env, (code))

int divide(int a, int b)
{
    if (b == 0) THROW(1);
    return a / b;
}

int main(void)
{
    TRY {
        printf("%d\n", divide(10, 2));
        printf("%d\n", divide(10, 0));
        printf("never reached\n");
    } CATCH {
        printf("caught division by zero\n");
    }
    return 0;
}
```

Why this is a bad idea in real code:

1. **Local variables are unreliable.** Any non-volatile local in the function that called `setjmp`, modified after the `setjmp`, has an indeterminate value after the jump — the compiler may have had it in a register that got clobbered. You must mark such variables `volatile`, and forgetting is silent.
2. **Nothing gets cleaned up.** Every `malloc`, every open `FILE *`, every held lock in the skipped frames leaks. C has no destructors, so there's no mechanism to unwind resources.
3. **One global jump target.** Nesting requires managing a stack of `jmp_buf`s yourself, and it isn't thread-safe unless the buffer is thread-local.
4. **longjmp into a function that has already returned is undefined**, and nothing warns you.

Real C error handling is return codes plus the `goto` cleanup ladder from 31.2. `setjmp` is for the rare cases like a recursive-descent parser bailing out of deep recursion, and even there people regret it.

Chapter 35 — Incomplete Types

35.1

```
/* ===== counter.h ===== */
#ifdef COUNTER_H
#define COUNTER_H

typedef struct counter Counter; /* incomplete: fields not visible */

Counter *counter_new(int start);
void counter_inc(Counter *c);
int counter_get(const Counter *c);
void counter_free(Counter *c);

#endif
```

```
/* ===== counter.c ===== */
#include <stdlib.h>
#include "counter.h"

struct counter { int value; int increments; }; /* complete only here */

Counter *counter_new(int start)
{
    Counter *c = malloc(sizeof *c);
    if (c) { c->value = start; c->increments = 0; }
    return c;
}

void counter_inc(Counter *c) { c->value++; c->increments++; }
int counter_get(const Counter *c) { return c->value; }
void counter_free(Counter *c) { free(c); }
```

```

/* ===== main.c ===== */
#include <stdio.h>
#include "counter.h"

int main(void)
{
    Counter *c = counter_new(10);
    if (!c) return 1;
    for (int i = 0; i < 5; i++) counter_inc(c);
    printf("%d\n", counter_get(c));
    /* c->value = 99;          ERROR: dereferencing pointer to incomplete type */
    /* Counter x;             ERROR: storage size of 'x' isn't known */
    /* sizeof(Counter);       ERROR: invalid application of sizeof */
    counter_free(c);
    return 0;
}

```

This is **the** encapsulation mechanism in C. Because `main.c` can't see the fields, it can't depend on them, so you can add, remove, or reorder members without recompiling callers — you get a stable ABI. `FILE` works exactly this way, which is why you always handle a `FILE *` and never a `FILE`.

The cost: every instance must be heap-allocated by the module, since callers can't know the size to put one on the stack.

35.2

```

#include <stdio.h>

struct b;                /* forward declaration */

struct a {
    int id;
    struct b *partner;    /* pointer to incomplete type: fine */
};

struct b {
    int id;
    struct a *partner;
    /* struct a whole;     ERROR: field has incomplete type */
};

int main(void)
{
    struct a x = {1, NULL};
    struct b y = {2, &x};
    x.partner = &y;
    printf("%d <-> %d\n", x.partner->id, y.partner->id);
    return 0;
}

```

A **pointer** to an incomplete type is always fine — every pointer is the same size, so the compiler doesn't need the layout. A **member by value** requires the complete type, and mutual containment by value is impossible anyway: the struct would have to contain itself, infinitely.

Chapter 36 — Complex Numbers

36.1

```

#include <stdio.h>
#include <complex.h>

int main(void)
{
    double complex a = 3.0 + 4.0 * I;
    double complex b = 1.0 - 2.0 * I;

    double complex sum = a + b;
    double complex prod = a * b;
    double complex quot = a / b;

    printf("a      = %.2f + %.2fi\n", creal(a), cimag(a));
    printf("a+b    = %.2f + %.2fi\n", creal(sum), cimag(sum));
    printf("a*b    = %.2f + %.2fi\n", creal(prod), cimag(prod));
    printf("a/b    = %.2f + %.2fi\n", creal(quot), cimag(quot));
    printf("|a|    = %.4f\n", cabs(a));
    printf("conj a = %.2f + %.2fi\n", creal(conj(a)), cimag(conj(a)));
    return 0;
}

```

Check by hand: $(3+4i)(1-2i) = 3 - 6i + 4i - 8i^2 = 3 - 2i + 8 = 11 - 2i$. And $|3+4i| = 5$.

Compile with `-lm`.

36.2

```

#include <stdio.h>
#include <complex.h>

int main(void)
{
    const int W = 78, H = 24, MAXIT = 100;
    const char *ramp = " .:-=+*#%@";

    for (int y = 0; y < H; y++) {
        for (int x = 0; x < W; x++) {
            double re = -2.2 + 3.0 * x / W;
            double im = -1.2 + 2.4 * y / H;
            double complex c = re + im * I, z = 0;

            int it = 0;
            while (it < MAXIT && cabs(z) <= 2.0) {
                z = z * z + c;
                it++;
            }
            putchar(it == MAXIT ? '@' : ramp[it * 9 / MAXIT]);
        }
        putchar('\n');
    }
    return 0;
}

```

Compile with `-lm`. In production you'd test `creal(z)*creal(z) + cimag(z)*cimag(z) <= 4.0` instead of `cabs(z) <= 2.0` — same test, no square root, several times faster.

Chapter 37 — Fixed Width Integer Types

37.1

```
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>

int main(void)
{
    printf("int8_t  %zu bytes  [%\" PRIu8 \" , \"%\" PRIu8 \" ]\\n\",
           sizeof(int8_t), INT8_MIN, INT8_MAX);
    printf("int16_t %zu bytes  [%\" PRIu16 \" , \"%\" PRIu16 \" ]\\n\",
           sizeof(int16_t), INT16_MIN, INT16_MAX);
    printf("int32_t %zu bytes  [%\" PRIu32 \" , \"%\" PRIu32 \" ]\\n\",
           sizeof(int32_t), INT32_MIN, INT32_MAX);
    printf("int64_t %zu bytes  [%\" PRIu64 \" , \"%\" PRIu64 \" ]\\n\",
           sizeof(int64_t), INT64_MIN, INT64_MAX);
    printf("uint32_t %zu bytes  [0, \"%\" PRIu32 \" ]\\n\", sizeof(uint32_t), UINT32_MAX);
    printf("uint64_t %zu bytes  [0, \"%\" PRIu64 \" ]\\n\", sizeof(uint64_t), UINT64_MAX);
    printf("intmax_t %zu bytes\\n\", sizeof(intmax_t));
    printf("intptr_t %zu bytes\\n\", sizeof(intptr_t));
    return 0;
}
```

The PRIu64 macros exist because int64_t is long on 64-bit Linux and long long on Windows — %ld is right on one and wrong on the other, and printf format mismatches are undefined behaviour. The macros expand to string literals which concatenate with the surrounding format at compile time. Ugly, portable, correct.

37.2

```

#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <string.h>

struct rec {
    int32_t id;
    int64_t timestamp;
    uint16_t flags;
    char    tag[16];
};

int main(void)
{
    struct rec out = { .id = 7, .timestamp = 1700000000LL, .flags = 0x00FF };
    snprintf(out.tag, sizeof out.tag, "hello");

    FILE *f = fopen("rec.bin", "wb");
    if (!f) { perror("rec.bin"); return 1; }
    fwrite(&out, sizeof out, 1, f);
    fclose(f);

    struct rec in;
    f = fopen("rec.bin", "rb");
    if (!f) { perror("rec.bin"); return 1; }
    fread(&in, sizeof in, 1, f);
    fclose(f);

    printf("%" PRIi32 " %" PRIi64 " %#x [%s]\n", in.id, in.timestamp, in.flags, in.tag);
    return 0;
}

```

Fixed-width types solve the problem that `int` might be 2 bytes on an embedded target and `long` 4 or 8 depending on the platform — so a file written by one build is misparsed by another.

What it does **not** solve: **endianness** (a little-endian x86 writes `0x01020304` as `04 03 02 01`, a big-endian machine reads it back as `0x04030201`), and **struct padding**, which is still compiler-dependent. Real portable formats serialize field by field into a byte buffer with an explicit byte order, rather than `fwrite`ing a struct.

Chapter 38 — Date and Time

38.1

```

#include <stdio.h>
#include <time.h>

int main(void)
{
    time_t now = time(NULL);
    if (now == (time_t)-1) { perror("time"); return 1; }

    struct tm *lt = localtime(&now);
    if (!lt) { perror("localtime"); return 1; }

    char buf[128];
    strftime(buf, sizeof buf, "%Y-%m-%dT%H:%M:%S%Z", lt);
    printf("ISO 8601 : %s\n", buf);

    strftime(buf, sizeof buf, "%A, %B %e, %Y at %I:%M %p", lt);
    printf("Long      : %s\n", buf);

    strftime(buf, sizeof buf, "%A", lt);
    printf("Weekday   : %s\n", buf);
    return 0;
}

```

`localtime` returns a pointer to a **static** buffer — the next call overwrites it, and it isn't thread-safe. Use `localtime_r` (POSIX) or `localtime_s` (C11 Annex K) in anything real.

38.2

```

#include <stdio.h>
#include <time.h>
#include <stdbool.h>

bool is_leap(int y) { return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0); }

const char *weekday_of(int year, int month, int day)
{
    static const char *names[] = {"Sunday", "Monday", "Tuesday", "Wednesday",
                                   "Thursday", "Friday", "Saturday"};

    struct tm t = {0};
    t.tm_year = year - 1900; /* years since 1900 */
    t.tm_mon = month - 1; /* 0-based month */
    t.tm_mday = day;
    t.tm_isdst = -1; /* "figure out DST yourself" */

    if (mktime(&t) == (time_t)-1) return "INVALID";
    return names[t.tm_wday];
}

double days_between(int y1, int m1, int d1, int y2, int m2, int d2)
{
    struct tm a = {0}, b = {0};
    a.tm_year = y1 - 1900; a.tm_mon = m1 - 1; a.tm_mday = d1; a.tm_isdst = -1;
    b.tm_year = y2 - 1900; b.tm_mon = m2 - 1; b.tm_mday = d2; b.tm_isdst = -1;
    return difftime(mktime(&b), mktime(&a)) / 86400.0;
}

int main(void)
{
    int years[] = {1900, 1996, 2000, 2020, 2100};
    for (size_t i = 0; i < sizeof years / sizeof years[0]; i++)
        printf("Feb 29 %d: leap=%d -> %s\n", years[i], is_leap(years[i]),
              weekday_of(years[i], 2, 29));

    printf("days from 2020-01-01 to 2024-01-01: %.0f\n",
          days_between(2020, 1, 1, 2024, 1, 1));
    return 0;
}

```

`mktime` **normalizes**: feed it February 29th of a non-leap year and it silently rolls over to March 1st, which is exactly how you cross-check your leap-year logic. It also fills in `tm_wday` and `tm_yday` for you — those are outputs, not inputs.

The `-1900` on the year and the 0-based month are the two mistakes everyone makes. `tm_mday`, inconsistently, is 1-based.

Chapter 39 — Multithreading

39.1

```

#include <stdio.h>
#include <threads.h>

int worker(void *arg)
{
    int id = *(int *)arg;
    for (int i = 0; i < 5; i++) printf("thread %d: message %d\n", id, i);
    return id * 100;
}

int main(void)
{
    thrd_t t1, t2;
    int id1 = 1, id2 = 2;

    if (thrd_create(&t1, worker, &id1) != thrd_success) return 1;
    if (thrd_create(&t2, worker, &id2) != thrd_success) return 1;

    int r1, r2;
    thrd_join(t1, &r1);
    thrd_join(t2, &r2);
    printf("returned %d and %d\n", r1, r2);
    return 0;
}

```

Compile with `-pthread`. The interleaving changes between runs and is not something you can rely on or reason about. Note `id1` and `id2` are separate variables — passing the address of one shared loop counter is the classic bug, because the loop may advance before the thread reads it.

If your libc lacks C11 `<threads.h>`, use POSIX `pthread_create/pthread_join`; the structure is identical.

39.2

```

#include <stdio.h>
#include <threads.h>

#define NTHREADS 4
#define NINCS    100000

static long unsafe_counter = 0;
static long safe_counter = 0;
static mtx_t lock;

int unsafe_worker(void *arg)
{
    (void)arg;
    for (int i = 0; i < NINCS; i++) unsafe_counter++;
    return 0;
}

int safe_worker(void *arg)
{
    (void)arg;
    for (int i = 0; i < NINCS; i++) {
        mtx_lock(&lock);
        safe_counter++;
        mtx_unlock(&lock);
    }
    return 0;
}

static void run(int (*fn)(void *))
{
    thrd_t t[NTHREADS];
    for (int i = 0; i < NTHREADS; i++) thrd_create(&t[i], fn, NULL);
    for (int i = 0; i < NTHREADS; i++) thrd_join(t[i], NULL);
}

int main(void)
{
    if (mtx_init(&lock, mtx_plain) != thrd_success) return 1;

    run(unsafe_worker);
    run(safe_worker);

    printf("expected: %d\n", NTHREADS * NINCS);
    printf("unsafe:   %ld\n", unsafe_counter);
    printf("safe:     %ld\n", safe_counter);

    mtx_destroy(&lock);
    return 0;
}

```

`counter++` is three operations — load, add, store — and two threads can interleave between them so one increment is lost. The unsafe total typically lands somewhere between 100,000 and 400,000 and differs every run.

The mutex version is correct but slow here, because the critical section is one instruction while lock/unlock is dozens. That's the setup for the atomics comparison next.

Chapter 40 — Atomics

40.1

```
#include <stdio.h>
#include <threads.h>
#include <stdatomic.h>

#define NTHREADS 4
#define NINCS 100000

static atomic_long counter = 0;

int worker(void *arg)
{
    (void)arg;
    for (int i = 0; i < NINCS; i++)
        atomic_fetch_add(&counter, 1);
    return 0;
}

int main(void)
{
#ifdef __STDC_NO_ATOMICS__
    printf("atomics not supported\n");
    return 0;
#else
    thrd_t t[NTHREADS];
    for (int i = 0; i < NTHREADS; i++) thrd_create(&t[i], worker, NULL);
    for (int i = 0; i < NTHREADS; i++) thrd_join(t[i], NULL);

    printf("expected %d, got %ld\n", NTHREADS * NINCS, atomic_load(&counter));
    return 0;
#endif
}
```

Compile with `-pthread`. Correct, and typically several times faster than the mutex: `atomic_fetch_add` compiles to a single `lock xadd` instruction rather than a lock acquire/release pair. Note `counter++` on an `atomic_long` is *also* atomic — the operators are overloaded for atomic types — but the explicit function makes the intent obvious to readers.

The tradeoff: atomics only protect one variable at a time. If you need two fields updated together as a unit, you still need a mutex.

40.2

```

#include <stdio.h>
#include <stdbool.h>
#include <threads.h>
#include <stdatomic.h>

static int payload = 0;           /* plain, non-atomic */
static atomic_bool ready = false;

int producer(void *arg)
{
    (void)arg;
    payload = 42;                 /* 1. write the data */
    atomic_store_explicit(&ready, true, memory_order_release); /* 2. publish */
    return 0;
}

int consumer(void *arg)
{
    (void)arg;
    while (!atomic_load_explicit(&ready, memory_order_acquire))
        thrd_yield();
    printf("consumer sees payload = %d\n", payload); /* guaranteed 42 */
    return 0;
}

int main(void)
{
    thrd_t p, c;
    thrd_create(&c, consumer, NULL);
    thrd_create(&p, producer, NULL);
    thrd_join(p, NULL);
    thrd_join(c, NULL);
    return 0;
}

```

Compile with `-pthread`. The release store guarantees that everything written *before* it in program order is visible to any thread that performs an acquire load which reads that value. That's what makes the plain, non-atomic `payload` safe to read.

With `memory_order_relaxed` on both sides you'd still get atomicity of the flag itself, but **no ordering guarantee at all** for `payload` — the compiler could hoist the store, or the CPU could reorder it after the flag, and the consumer could legally print 0. Relaxed ordering is only for things like statistics counters where you want the count to be exact but don't care when anyone sees it.

Chapter 41 — Function Specifiers and Alignment

41.1

```

/* ===== util.h ===== */
#ifndef UTIL_H
#define UTIL_H
#include <stdnoreturn.h>

inline int square(int x) { return x * x; } /* inline DEFINITION */

noreturn void die(const char *msg);

#endif

```

```

/* ===== util.c ===== */
#include <stdio.h>
#include <stdlib.h>
#include "util.h"

extern inline int square(int x); /* emits exactly one external definition */

noreturn void die(const char *msg)
{
    fprintf(stderr, "fatal: %s\n", msg);
    exit(EXIT_FAILURE);
}

```

```

/* ===== main.c ===== */
#include <stdio.h>
#include "util.h"

int check(int x)
{
    if (x < 0) die("negative input");
    return square(x); /* compiler knows die() never returns */
}

int main(void)
{
    printf("%d\n", check(7));
    return 0;
}

```

C's `inline` rules are genuinely awkward: an `inline` definition in a header is *not* an external definition, so if the compiler decides not to inline a call, you get an undefined reference. Exactly one `.c` file must say `extern inline int square(int x);` to emit the real one. Most projects sidestep all of this by writing `static inline` in the header, which gives each translation unit a private copy.

`noreturn` on `die` is what stops “control reaches end of non-void function” in `check`, and lets the optimizer drop everything after the call.

41.2

```

#include <stdio.h>
#include <stdalign.h>
#include <stdint.h>

struct mixed { char c; double d; int i; };

int main(void)
{
    printf("alignof(char)   = %zu\n", alignof(char));
    printf("alignof(int)    = %zu\n", alignof(int));
    printf("alignof(double) = %zu\n", alignof(double));
    printf("alignof(void *) = %zu\n", alignof(void *));
    printf("alignof(struct mixed) = %zu, sizeof = %zu\n",
          alignof(struct mixed), sizeof(struct mixed));

    alignas(64) unsigned char buf[128];
    uintptr_t addr = (uintptr_t)buf;
    printf("buf at %p, low 6 bits = %lx, 64-aligned: %s\n",
          (void *)buf, (unsigned long)(addr & 63),
          (addr & 63) == 0 ? "yes" : "no");

    return 0;
}

```

A struct's alignment is that of its most-demanding member, which is why `struct mixed` is 8-aligned and 24 bytes rather than 13.

The practical use of `alignas(64)`: 64 bytes is the cache line size on most x86-64 machines. Aligning per-thread data to a cache line prevents **false sharing** — two threads writing to different variables that happen to share a line, forcing the line to bounce between cores. It's a common and very real cause of a "parallel" program running slower than the serial one.

Milestone Project Notes

Full solutions aren't given for the milestones — the point is that you architect them. Some hints on the parts that actually trip people:

Milestone A (word frequency). Use your `read_file` from 12.4, then walk the buffer in place: instead of copying each word out, record a `char *` start and a length. Convert to lowercase and strip punctuation as you go. A dynamic array of `struct { char *word; size_t len; int count; }` with a linear search is fine at first — it's $O(n^2)$ and slow on a large book, which is exactly the motivation for Milestone B.

Milestone B (hash table). FNV-1a is the hash to use: start at `2166136261u`, and for each byte `h ^= byte; h *= 16777619u;`. Keep the table size a power of two so the index is `h & (cap - 1)` instead of a modulo. Resize by allocating a new bucket array and rehashing everything — you cannot just copy the buckets, since the index depends on the capacity. Store the hash in each node so resizing doesn't have to rehash the strings.

Milestone C (contact manager). The hard part is CSV parsing, not the data structure. Decide up front how you'll handle quoted fields containing commas, and write that as a state machine rather than `strtok` — `strtok` modifies its input, isn't reentrant, and collapses consecutive delimiters, which silently drops your empty fields.

Milestone D. Whatever you pick, the deliverable is: a Makefile, a README explaining what it does and how to build it, tests you can run with one command, and a clean run under `-fsanitize=address,undefined`. That last item is what makes a co-op reviewer take the repo seriously.